

DOCUMENTATION TECHNIQUE

Serveur de ticketing GLPI sécurisé

Installation, sécurisation, exploitation et reprise du service

Auteur	Maël Jansen, Kilian Levasseur
Date	4 septembre 2026
Serveur	<code>ubuntuserver2204</code> – Ubuntu Server 22.04 LTS
Adresse locale	192.168.1.17 (<code>enp0s8</code>)
Applicatif	GLPI 10 – MariaDB 10.11 – NGINX, sous Docker Compose
Accès au service	HTTPS, uniquement depuis le tunnel VPN
Référentiel	Cahier de recettes du projet
Dépôt	systeme-de-ticketing-securise

Sommaire

Sommaire.....	2
1. Présentation du service.....	5
1.1 Objet et périmètre.....	5
1.2 Architecture.....	5
1.3 Inventaire de la plateforme.....	5
1.4 Composants applicatifs et persistance.....	6
1.5 Matrice des flux réseau.....	6
1.6 Arborescence des fichiers.....	7
1.7 Le principe retenu : la défense en profondeur.....	8
2. Procédure d'installation.....	9
2.1 Prérequis.....	9
2.2 Mise à jour du socle.....	9
2.3 Installation de Docker.....	9
2.4 Arborescence et secrets.....	10
2.5 Fichier de composition.....	10
2.6 Premier démarrage.....	11
2.7 Assistant d'installation de GLPI.....	12
2.8 Durcissement immédiat des comptes.....	13
2.9 Contrôles de fin d'installation.....	15
2.10 Collecteur de courriels : création automatisée des tickets.....	16
2.10.1 Boîte de messagerie dédiée.....	16
2.10.2 Déclaration du collecteur.....	16
2.10.3 Reconnaissance de l'expéditeur.....	17
2.10.4 Planification de la relève.....	18
2.10.5 Recette du collecteur.....	19
2.10.6 Notifications sortantes.....	20
2.10.7 Dépannage.....	21
3. Procédure de sécurisation.....	21
3.1 Certificat TLS.....	21
3.2 Reverse proxy NGINX.....	22
3.3 Bascule en HTTPS et contrôles.....	24
3.4 Tunnel OpenVPN.....	25

3.4.1 Installation du serveur.....	25
3.4.2 Créer un accès client.....	26
3.4.3 Connexion et révocation.....	26
3.4.4 Variante : mise en place manuelle de la PKI avec Easy-RSA.....	26
3.5 Pare-feu UFW.....	29
3.6 Isolation des conteneurs : la chaîne DOCKER-USER.....	30
3.6.1 Rendre la règle persistante — correctif prioritaire.....	31
3.7 Durcissement de GLPI.....	32
3.8 Détection et prévention d'intrusion : Suricata.....	33
3.8.1 Où placer la sonde : la vraie difficulté.....	33
3.8.2 Relever les interfaces et le sous-réseau Docker.....	33
3.8.3 Installation.....	34
3.8.4 Configuration : les trois blocs qui comptent.....	34
3.8.5 Valider avant de démarrer.....	35
3.8.6 Jeux de règles et recette de la détection.....	35
3.8.7 Passage en mode prévention.....	36
3.8.8 Dépannage.....	37
4. Exploitation courante.....	37
4.1 Commandes de base.....	37
4.2 Journalisation et diagnostic.....	38
4.3 Sauvegarde.....	38
4.4 Restauration.....	39
4.5 Renouvellement du certificat TLS.....	40
4.6 Mises à jour.....	41
5. État de conformité au cahier de recettes.....	42
5.1 Tableau de conformité.....	42
5.2 Poste client et agent d'inventaire.....	42
5.3 Collecte des courriels et création automatisée de tickets.....	43
5.4 Outil de contrôle à distance.....	43
5.5 Double authentification.....	43
5.6 Système de détection et de prévention d'intrusion Suricata.....	44
6. Registre des points de vigilance.....	45
Annexe A. docker-compose.yml.....	47
Annexe B. nginx.conf.....	47

Annexe C. Jeu de règles réseau de référence.....	48
Annexe D. Collecteur de courriels : planification et contrôles.....	48
Annexe E. Suricata : jeu de règles local.....	49
Annexe F. Table des figures.....	50

Ce document est la référence unique du serveur de ticketing. Il s'adresse à toute personne appelée à l'installer, le sécuriser, l'exploiter ou le reconstruire. Les procédures des chapitres 2 et 3 sont complètes et reproductibles depuis un système nu, et illustrées par les captures relevées lors du déploiement initial : pour chaque étape, la commande, le résultat effectivement obtenu et le contrôle à effectuer. Toutes les commandes sont données telles qu'exécutées sur Ubuntu Server 22.04. Cette révision ajoute le collecteur de courriels, en service et illustré (paragraphe 2.10), la mise en place manuelle de la PKI du tunnel (paragraphe 3.4.4) et la procédure de détection et de prévention d'intrusion Suricata, établie mais non encore déployée (paragraphe 3.8).

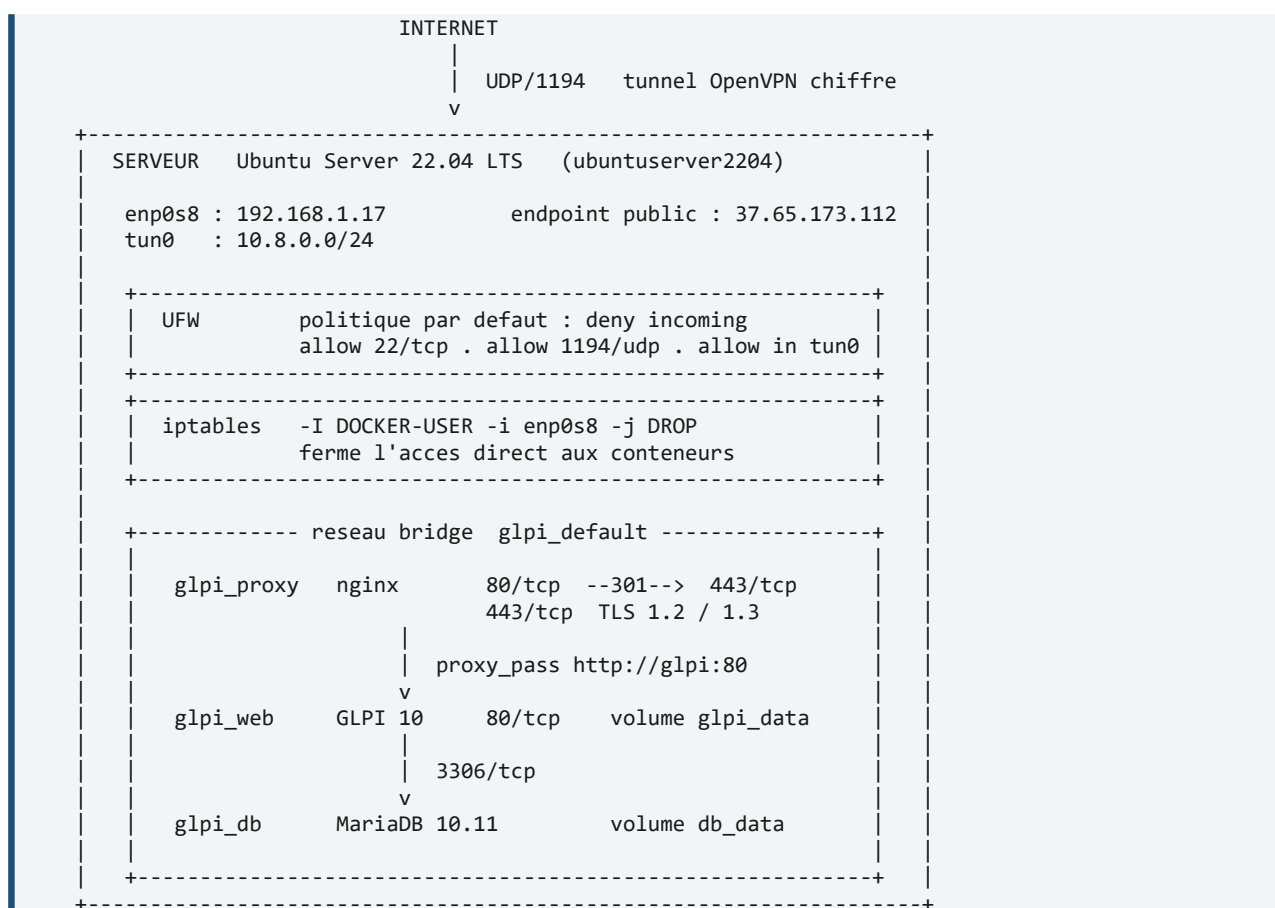
1. Présentation du service

1.1 Objet et périmètre

Le service met à disposition un outil de gestion de tickets et de parc informatique, GLPI, à destination des utilisateurs et des techniciens. Le cahier de recettes du projet ne demande pas seulement de l'installer, mais de le **rendre inaccessible autrement que par un tunnel VPN**. Deux de ses remarques commandent l'ensemble de l'architecture décrite ici : « *Attention à sécuriser le tunnel* » et « *N'autoriser que le tunnel du VPN en entrée pour passer* ».

Cette contrainte n'est pas un détail de configuration : elle explique pourquoi le port 443 n'est pas ouvert dans le pare-feu, pourquoi une règle **iptables** particulière est nécessaire, et pourquoi certaines options courantes – TeamViewer, par exemple – sont écartées au chapitre 5. Toute intervention sur ce serveur doit la préserver.

1.2 Architecture



Architecture du service : un seul point d'entrée, le tunnel VPN.

1.3 Inventaire de la plateforme

Élément	Valeur	Remarque
Système d'exploitation	Ubuntu Server 22.04 LTS (jammy)	Support jusqu'en avril 2027
Nom d'hôte	ubuntuserver2204	
Utilisateur d'administration	ubuntu	Élévation par sudo

Interface physique	enp0s8 – 192.168.1.17	Réseau local
Interface du tunnel	tun0 – 10.8.0.0/24	Créée par OpenVPN ; le serveur prend 10.8.0.1
Adresse publique	37.65.173.112	Point d'entrée du tunnel
Moteur de conteneurs	docker.io (dépôt Ubuntu)	
Orchestration	docker-compose 1.29.2 et docker compose v2	Les deux sont présents ; utiliser `docker compose` (v2)
Mémoire requise	4 Go minimum	Exigence du cahier de recettes
Répertoire de travail	/home/ubuntu/ticketing-projet	

1.4 Composants applicatifs et persistance

Conteneur	Image	Rôle	Ports publiés	Volume monté
glpi_proxy	nginx:latest	Terminaison TLS, redirection 80 vers 443, reverse proxy	80, 443	nginx.conf et certs/ en lecture seule
glpi_web	diouxx/glpi:latest	Application GLPI 10	aucun	glpi_data sur /var/www/html/glpi
glpi_db	mariadb:10.11	Base de données glpidb	aucun	db_data sur /var/lib/mysql

Le fait que **glpi_web** et **glpi_db** ne publient **aucun** port est structurant : même si le filtrage réseau venait à tomber, l'application et la base resteraient inatteignables autrement que par le proxy.

Réseau Docker : **glpi_default**, de type bridge, créé automatiquement par Compose. Les conteneurs s'y résolvent par leur **nom de service** – **db**, **glpi**, **nginx** – et non par leur nom de conteneur. C'est pourquoi **nginx.conf** relaie vers **http://glpi:80**.

Volume Docker	Contenu	Conséquence en cas de perte
glpi_db_data	Données MariaDB : parc, tickets, utilisateurs, droits	Perte totale de l'historique
glpi_glpi_data	Fichiers GLPI : documents joints, greffons, configuration locale	Perte des pièces jointes des tickets

Nom réel des volumes

Compose préfixe les volumes par le nom du répertoire du projet. Le fichier de composition étant dans **~/ticketing-projet/glpi/**, les volumes s'appellent **glpi_db_data** et **glpi_glpi_data** – et non **db_data** et **glpi_data**. C'est sous ces noms qu'il faut les désigner dans **docker volume ls** et dans les procédures de sauvegarde du paragraphe 4.3.

1.5 Matrice des flux réseau

Cette matrice décrit l'état attendu du filtrage. C'est la référence à consulter lors de tout dépannage d'accès.

#	Source	Destination	Port	Décision et règle applicable
---	--------	-------------	------	------------------------------

1	Internet	Serveur, <code>enp0s8</code>	1194/udp	Autorisé – <code>ufw allow 1194/udp</code>
2	Poste d'administration	Serveur, <code>enp0s8</code>	22/tcp	Autorisé – <code>ufw allow 22/tcp</code>
3	Client du tunnel, 10.8.0.0/24	<code>glpi_proxy</code>	443/tcp	Autorisé – <code>ufw allow in on tun0</code> ; la règle <code>DOCKER-USER</code> ne s'applique pas, l'interface d'entrée étant <code>tun0</code>
4	Client du tunnel	<code>glpi_proxy</code>	80/tcp	Autorisé puis redirigé en 301 vers 443/tcp par NGINX
5	Réseau local, 192.168.1.0/24	Conteneurs	tout	Rejeté – <code>DOCKER-USER -i enp0s8 -j DROP</code> . C'est la règle qui applique la consigne du projet.
6	Réseau local	Serveur, hors 22 et 1194	tout	Rejeté – <code>ufw default deny incoming</code>
7	<code>glpi_proxy</code>	<code>glpi_web</code>	80/tcp	Autorisé – réseau <code>glpi_default</code>
8	<code>glpi_web</code>	<code>glpi_db</code>	3306/tcp	Autorisé – réseau <code>glpi_default</code>
9	Serveur et conteneurs	Internet	tout	Autorisé – <code>ufw default allow outgoing</code> (mises à jour, relève IMAPS en 993/tcp du collecteur de courriels, paragraphe 2.10)

Le port 443 n'est pas ouvert dans UFW, et c'est voulu

L'absence de `ufw allow 443/tcp` n'est pas un oubli. Le service ne doit être joignable que depuis le tunnel, où c'est la règle `allow in on tun0` qui l'autorise. **Ne pas ouvrir 443 pour dépanner un accès** : cela annulerait l'ensemble du dispositif. Si un client ne parvient pas à joindre GLPI, vérifier d'abord que son tunnel est monté – voir l'arbre de diagnostic du paragraphe 4.2.

1.6 Arborescence des fichiers

```

/home/ubuntu/ticketing-projet/
|
+-- glpi/
|   +-- .env                secrets MariaDB    (600, jamais versionne)
|   +-- docker-compose.yml  definition des trois services
|
+-- nginx/
|   +-- nginx.conf          reverse proxy et terminaison TLS
|   +-- certs/
|       +-- glpi.crt        certificat auto-signé    (644)
|       +-- glpi.key        cle privée                (600, root)

```

Chemins relatifs du fichier de composition

`docker-compose.yml` monte `../nginx/nginx.conf` et `../nginx/certs`. Ces chemins sont relatifs au répertoire du fichier, soit `~/ticketing-projet/glpi/` : ils sont donc **corrects sur le serveur**. Attention en revanche dans le dépôt

Git, où les deux fichiers sont regroupés dans un unique répertoire `serveur/` : y déplier l'arborescence ci-dessus avant de lancer la pile.

1.7 Le principe retenu : la défense en profondeur

La sécurisation ne repose pas sur une mesure unique mais sur quatre couches indépendantes. Chacune répond à une menace précise, et la défaillance de l'une ne suffit pas à compromettre le service. Cette grille sert de fil conducteur au chapitre 3 et de référence lors de tout audit.

Couche	Mécanisme	Menace couverte	Procédure
1. Périmètre réseau	OpenVPN en 1194/udp, authentification par certificat (PKI)	Exposition directe du service de ticketing sur Internet	3.4
2. Filtrage réseau	UFW en <code>deny incoming</code> + règle <code>DOCKER-USER</code> bloquant <code>enp0s8</code>	Accès au service depuis le réseau local sans passer par le tunnel	3.5, 3.6
3. Transport	Certificat X.509, TLS 1.2 et 1.3, redirection 301 de HTTP vers HTTPS	Interception des identifiants et des données circulant en clair	3.1 à 3.3
4. Application	Mots de passe, double authentification, politique de mot de passe	Comptes par défaut de GLPI, publiquement documentés	2.8, 3.7
5. Détection et prévention	Suricata en IDS puis IPS, à l'écoute de <code>enp0s8</code> , <code>tun0</code> et du réseau des conteneurs	Ce qui est tenté malgré les quatre couches précédentes : balayages, injections, force brute	3.8

2. Procédure d'installation

Cette procédure installe le service depuis un Ubuntu Server 22.04 fraîchement déployé. Elle correspond à ce qui a été exécuté lors du déploiement initial, dont les captures constituent le résultat attendu de chaque étape. À l'issue du chapitre, GLPI est fonctionnel mais **encore accessible en HTTP et sans restriction réseau** : le chapitre 3 est indissociable de celui-ci.

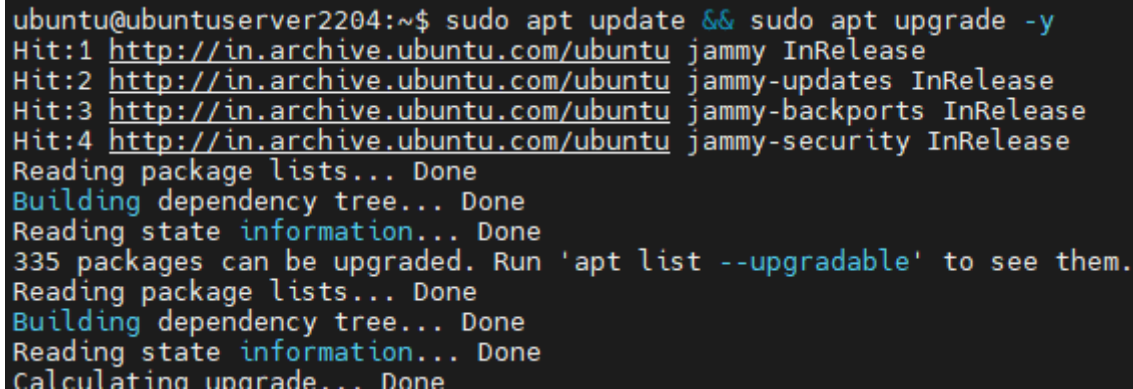
2.1 Prérequis

- Une machine virtuelle Ubuntu Server 22.04 LTS, **4 Go de RAM minimum**, avec accès administrateur **sudo**.
- Une interface réseau atteignable depuis le réseau local (ici **enp0s8**, 192.168.1.17) et une adresse publique pour le point d'entrée du tunnel.
- Un accès Internet sortant pour récupérer les paquets et les images de conteneurs.
- Un accès console à la machine, ou une seconde session SSH ouverte : plusieurs étapes du chapitre 3 manipulent le pare-feu et peuvent couper la session en cours.

2.2 Mise à jour du socle

Premier réflexe avant toute installation : appliquer les correctifs de sécurité en attente. Installer un service à sécuriser sur un système obsolète reviendrait à verrouiller la porte d'entrée en laissant les fenêtres ouvertes.

```
sudo apt update && sudo apt upgrade -y
sudo reboot # uniquement si un nouveau noyau a été installé
```

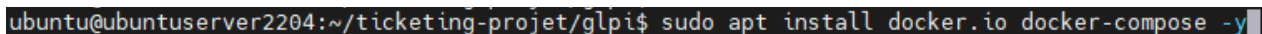


```
ubuntu@ubuntuserver2204:~$ sudo apt update && sudo apt upgrade -y
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
335 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
```

Figure 1 – Résultat attendu : la liste des paquets est actualisée et la mise à niveau démarre. Lors du déploiement initial, 335 paquets étaient en attente.

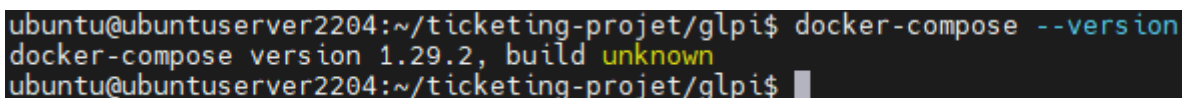
2.3 Installation de Docker

```
sudo apt install docker.io docker-compose -y
sudo systemctl enable --now docker
docker-compose --version
```



```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo apt install docker.io docker-compose -y
```

Figure 2 – Installation du moteur Docker et de Docker Compose.



```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ docker-compose --version
docker-compose version 1.29.2, build unknown
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$
```

Figure 3 – Contrôle de la version. La commande doit répondre sans erreur ; la version 1.29.2 est celle des dépôts Ubuntu 22.04.

Optionnel, pour éviter `sudo` devant chaque commande Docker. La session doit être rouverte pour que l'appartenance au groupe prenne effet :

```
sudo usermod -aG docker $USER
```

Implication de sécurité

Appartenir au groupe `docker` équivaut à disposer des droits `root` sur la machine : un conteneur peut monter le système de fichiers de l'hôte. **N'y ajouter aucun compte de service ni utilisateur non administrateur.**

2.4 Arborescence et secrets

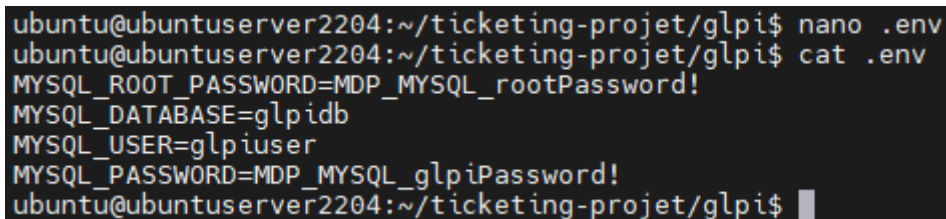
```
mkdir -p ~/ticketing-projet/glpi ~/ticketing-projet/nginx/certs
cd ~/ticketing-projet/glpi
nano .env
```

Les identifiants de la base sont placés dans un fichier `.env` que Docker Compose charge automatiquement. L'intérêt : `docker-compose.yml` peut être versionné et partagé, alors que `.env` reste local. Les mots de passe doivent être **générés, non choisis** :

```
MYSQL_ROOT_PASSWORD=<sortie de : openssl rand -base64 24>
MYSQL_DATABASE=glpidb
MYSQL_USER=glpiuser
MYSQL_PASSWORD=<sortie de : openssl rand -base64 24>

chmod 600 .env
echo '.env' >> ~/ticketing-projet/.gitignore
```

Le fichier `.env` ne doit être lisible que par son propriétaire, et jamais versionné.



```
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano .env
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat .env
MYSQL_ROOT_PASSWORD=MDP_MYSQL_rootPassword!
MYSQL_DATABASE=glpidb
MYSQL_USER=glpiuser
MYSQL_PASSWORD=MDP_MYSQL_glpiPassword!
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$
```

Figure 4 – Structure du fichier `.env` telle que créée lors du déploiement initial.

Point de vigilance

La capture ci-dessus affiche les mots de passe en clair. C'est acceptable sur un environnement de laboratoire éphémère, mais un `.env` ne doit jamais être versionné ni capturé sur un système réel. Les valeurs employées lors du déploiement initial doivent être considérées comme divulguées et **régénérées** – voir l'entrée 10 du registre du chapitre 6. Publier uniquement un `.env.example` sans valeurs.

2.5 Fichier de composition

Créer `~/ticketing-projet/glpi/docker-compose.yml`. À ce stade de la procédure, il ne déclare que deux services – la base et l'application – et publie le port 80 pour permettre l'accès à l'assistant d'installation.

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano docker-compose.yml
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat docker-compose.yml
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    ports:
      - "80:80"
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

volumes:
  db_data:
  glpi_data:
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ █

```

Figure 5 – Fichier de composition initial : MariaDB et GLPI. Noter la section ports du service glpi, qui sera retirée au paragraphe 3.2.

Reconstruction depuis zéro : deux étapes ou une seule ?

Le déploiement initial s'est fait en deux temps – installer en HTTP, puis placer le proxy devant – et cette documentation suit le même ordre pour que chaque capture corresponde à son étape. **Pour une reconstruction, il est préférable d'appliquer directement le fichier complet de l'annexe A**, puis de dérouler le paragraphe 3.1 (certificat) avant de lancer la pile : l'assistant d'installation de GLPI fonctionne aussi bien à travers le proxy HTTPS, et le service n'est alors jamais exposé en clair, même temporairement.

2.6 Premier démarrage

```

cd ~/ticketing-projet/glpi
sudo docker compose up -d
sudo docker compose ps

```

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo docker-compose up -d
Creating network "glpi_default" with the default driver
Creating volume "glpi_db_data" with default driver
Creating volume "glpi_glpi_data" with default driver
Pulling db (mariadb:10.11)...
10.11: Pulling from library/mariadb
0bf9ac32a939: Pull complete
92ff57b1eb9b: Pull complete
d544298cabd5: Pull complete
c0b110f56249: Pull complete
3c10d7f0cd20: Pull complete
38a738597cc9: Pull complete
8bee7ee7d49c: Pull complete
758b07585e0a: Pull complete
f8b370294f64: Download complete
db02198483d1: Download complete
Digest: sha256:ce66c7be32a03aabe7241d0a10993a2db827ef652a35d25727d92a832ac8ef73
Status: Downloaded newer image for mariadb:10.11
Pulling glpi (diouxx/glpi:latest)...
latest: Pulling from diouxx/glpi
0bb5c2a7dd90: Pull complete
9821c61dc279: Pull complete
fea1432adf09: Pull complete
b9347fe14756: Pull complete
ece6a8fdeb39: Download complete
Digest: sha256:9ca43af63ddcb7e1c9b7b452a50b8204980ac549526dee1c00296793bfe7789f
Status: Downloaded newer image for diouxx/glpi:latest
Creating glpi_db ... done
Creating glpi_web ... done

```

Figure 6 – Résultat attendu : création du réseau et des volumes, téléchargement des images, puis démarrage des conteneurs.

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ sudo docker-compose ps
      Name                    Command                                State      Ports
-----
glpi_db      docker-entrypoint.sh mariabd         Up          3306/tcp
glpi_web     /opt/glpi-start.sh                  Up          443/tcp, 0.0.0.0:80->80/tcp, :::80->80/tcp
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$

```

Figure 7 – Les deux conteneurs sont à l'état Up. glpi_db n'expose 3306 qu'en interne, tandis que glpi_web publie 0.0.0.0:80 – situation corrigée au paragraphe 3.2.

2.7 Assistant d'installation de GLPI

Ouvrir <http://<adresse du serveur>/> et dérouler l'assistant en six étapes. Paramètres de connexion à la base, à saisir tels quels :

Champ de l'assistant	Valeur	Origine
Serveur SQL	db	Nom du service dans le fichier de composition, pas son nom de conteneur
Utilisateur SQL	glpiuser	MYSQL_USER du fichier .env
Mot de passe SQL	valeur de MYSQL_PASSWORD	Fichier .env
Base de données	glpidb	MYSQL_DATABASE du fichier .env



Figure 8 – Étape 6 : l'installation est terminée. GLPI énumère les quatre comptes créés par défaut.

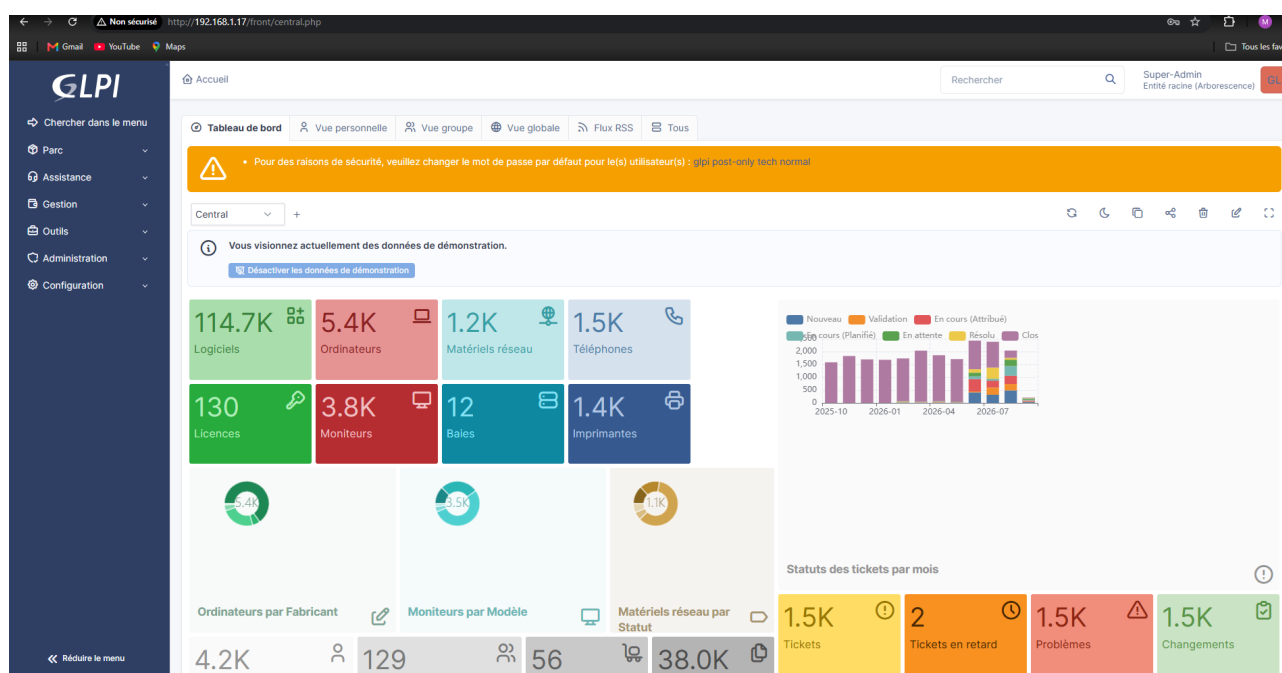


Figure 9 – Première connexion. Trois points à traiter sans délai : l'accès est en HTTP, le navigateur signale « Non sécurisé », et GLPI réclame le changement des mots de passe de glpi, post-only, tech et normal.

2.8 Durcissement immédiat des comptes

Les quatre comptes créés par l'assistant constituent la première vulnérabilité concrète du déploiement : leurs identifiants figurent dans la documentation publique de GLPI et sont testés systématiquement par les outils de balayage automatisé.

Compte	Mot de passe par défaut	Profil	État au déploiement initial
glpi	glpi	Super-Admin	Traité – voir figure 10
tech	tech	Technicien	À traiter
normal	normal	Normal	À traiter

post-only	postonly	Self-Service	À traiter

Depuis *Administration > Utilisateurs*, pour chacun des quatre comptes :

Figure 10 – Changement du mot de passe du compte Super-Admin glpi.

Deux autres actions relèvent du même moment :

```
# 1. désactiver les données de démonstration
#   depuis le tableau de bord : bouton "Désactiver les données de démonstration"
#   (sinon 5 400 ordinateurs et 1 500 tickets fictifs faussent tout indicateur)

# 2. supprimer le fichier d'installation, qui reste accessible par le web
sudo docker exec glpi_web rm -f /var/www/html/glpi/install/install.php
```

Pourquoi supprimer `install.php`

Tant que ce fichier est présent, l'assistant d'installation reste joignable par le web et permet, dans certaines configurations, de réinitialiser la connexion à la base. Sa suppression est une étape standard de durcissement de GLPI, à refaire après chaque mise à jour de l'image puisqu'elle porte sur le contenu du conteneur.

2.9 Contrôles de fin d'installation

```
cd ~/ticketing-projet/glpi

# les deux conteneurs sont a l'etat Up
sudo docker compose ps

# l'application repond
curl -I http://localhost/

# la base accepte les connexions
sudo docker exec glpi_db mariadb-admin ping
```

2.10 Collecteur de courriels : création automatisée des tickets

Le cahier de recettes demande « l'intégration d'un script pour collecter les mails vers GLPI pour la création automatisée des tickets » et invite à « voir les scripts qui existent déjà ». La réponse tient dans cette dernière remarque : le mécanisme est natif. GLPI dispose de ses propres collecteurs, qui relèvent une boîte de messagerie à intervalle régulier et transforment chaque message en ticket — objet du message en titre, corps en description, expéditeur reconnu comme demandeur. C'est cette voie qui a été retenue : **aucun script spécifique n'a été écrit**.

Pourquoi ne pas avoir écrit le script demandé

Un collecteur maison imposerait de gérer soi-même le décodage des messages et de leurs pièces jointes, le marquage de ceux déjà traités, et surtout le rattachement d'une réponse au ticket d'origine plutôt que la création d'un doublon. Le collecteur de GLPI fait tout cela et se configure depuis l'interface. **Configurer plutôt que réécrire est ici le choix le plus robuste**, et non un raccourci : c'est aussi celui que le cahier de recettes suggère en renvoyant aux scripts existants.

2.10.1 Boîte de messagerie dédiée

Un compte de messagerie dédié au support a été créé, tpticketing20@gmail.com, et non un compte nominatif. La raison est directe : **le collecteur détient les identifiants de la boîte qu'il relève**, enregistrés dans la base de GLPI. Y placer une boîte personnelle reviendrait à confier une messagerie privée à l'application.

Paramètre	Valeur retenue	Remarque
Serveur IMAP	imap.gmail.com	Boîte externe ; la politique <code>default allow outgoing</code> du pare-feu autorise déjà cette sortie
Port	993	IMAPS. Le port 143, en clair, est à écarter
Chiffrement	SSL/TLS	Le transport de la relève est chiffré
Identifiant	tpticketing20@gmail.com	Compte dédié au support
Authentification	Mot de passe d'application	Le mot de passe habituel du compte est refusé dès lors que la double authentification est active côté fournisseur

L'accès IMAP doit être activé dans les paramètres du compte, et un **mot de passe d'application** généré. C'est le point qui bloque le plus souvent : GLPI signale un échec d'authentification alors que les identifiants sont exacts, simplement parce que le fournisseur refuse le mot de passe principal sur un accès IMAP.

2.10.2 Déclaration du collecteur

Dans l'interface : **Configuration > Collecteurs > Ajouter**. Le collecteur créé porte le nom [Collecteur gmail](#) et l'identifiant interne 1.

Champ	Valeur	Pourquoi
Nom	Collecteur gmail	Libre ; sert d'étiquette dans les journaux de l'action automatique
Activé	Oui	Un collecteur désactivé est ignoré par mailgate , sans message d'erreur
Serveur / Port	imap.gmail.com / 993	IMAPS
Options de connexion	IMAP, SSL	Voir la réserve sur NO-VALIDATE-CERT ci-dessous
Identifiant	tpticketing20@gmail.com	Compte dédié
Mot de passe	Mot de passe d'application	GLPI ne le réaffiche jamais ; la case <i>Effacer</i> sert à le retirer

Dossier des messages entrants	Vide	Équivaut à INBOX
-------------------------------	------	-------------------------

GLPI construit à partir de ces champs la **chaîne de connexion** affichée sous le formulaire :

{imap.gmail.com:993/imap/ssl/novalidate-cert}. C'est elle qu'il faut lire pour contrôler la configuration réelle du collecteur — elle résume en une ligne le serveur, le port, le protocole et les options effectivement appliquées.

Figure 19 – Fiche du collecteur « Collecteur gmail ». La chaîne de connexion {imap.gmail.com:993/imap/ssl/novalidate-cert} résume la configuration effective ; le champ Mot de passe n'est jamais réaffiché par GLPI, ce qui explique qu'il apparaisse vide.

L'option novalidate-cert est à retirer

La chaîne de connexion de la figure 19 comporte **novalidate-cert** : GLPI ouvre bien une session chiffrée, mais **ne vérifie pas le certificat du serveur de messagerie**. Cette option n'a de justification que face à un serveur interne à certificat auto-signé. Le fournisseur employé ici présentant un certificat signé par une autorité publique, elle n'apporte aucune souplesse et retire une garantie : le chiffrement subsiste, l'authentification du serveur disparaît.

Correction : décocher **NO-VALIDATE-CERT** dans les options de connexion, enregistrer, puis contrôler que la relève fonctionne toujours en relançant l'action automatique. Voir l'entrée 11 du registre du chapitre 6.

2.10.3 Reconnaissance de l'expéditeur

C'est le réglage qui décide si un message devient un ticket ou disparaît. Si l'adresse de l'expéditeur ne correspond à aucun compte GLPI, le message est refusé et **aucun ticket n'est créé** — sans que l'expéditeur en soit averti. Le symptôme est déroutant : la relève fonctionne, les journaux montrent des messages traités, et la liste des tickets reste vide.

- **Retenu** — renseigner l'adresse de messagerie sur la fiche de chaque utilisateur (**Administration > Utilisateurs**, onglet *Éléments principaux*). L'expéditeur est alors reconnu comme demandeur du ticket : le suivi et les notifications reviennent à la bonne personne. C'est ce que vérifie la figure 21, où le demandeur est l'utilisateur JANSEN.
- **Alternative** — autoriser les tickets provenant d'utilisateurs anonymes (**Configuration > Générale**, onglet *Assistance*). Commode pour une démonstration, mais à écarter en exploitation : toute adresse connaissant la boîte peut alors ouvrir des tickets, sans demandeur identifiable.

2.10.4 Planification de la relève

La relève est portée par l'action automatique **mailgate** — *Récupération des messages (collecteurs)* — dans **Configuration > Actions automatiques**.

Réglage	Valeur	Justification
Statut	Programmée	Seul statut sous lequel le planificateur prend la tâche en compte
Mode d'exécution	CLI	Voir l'encadré ci-dessous
Fréquence d'exécution	5 minutes	Compromis entre réactivité perçue par le demandeur et charge sur la boîte relevée
Plage horaire d'exécution	0 → 24	Relève permanente ; une plage restreinte retarderait les tickets émis la nuit
Nombre de mails à traiter	10	Limite par passage : un afflux de messages ne bloque pas la tâche sur une seule exécution
Conservation des journaux	30 jours	Trace de chaque passage, exploitable pour établir qu'un message a bien été relevé

Accueil / Configuration / Actions automatiques

Rechercher

Super-Admin
Entité racine (Arborescence)

Action automatique - mailgate - ID 9

Actions 7/20

Action automatique

Statistiques

Journaux 10

Historique 2

Tous

Nom mailgate

Commentaires

Description Récupération des messages (collecteurs)

Fréquence d'exécution 5 minutes

Statut Programmée

Mode d'exécution CLI

Plage horaires d'exécution 0 → 24

Temps de conservation des journaux (en jours) 30

Nombre de mails à traiter 10

Dernière exécution 2026-09-04 12:02

Prochaine exécution 2026-09-04 12:07

Exécuter

Sauvegarder

Figure 20 – L'action automatique **mailgate**, en mode CLI et à une fréquence de cinq minutes. Les champs **Dernière exécution** (12:02) et **Prochaine exécution** (12:07) montrent que la tâche est réellement prise en charge par le planificateur.

Pourquoi le mode CLI, et non le mode GLPI

En mode « GLPI », les actions automatiques ne se déclenchent qu'à l'occasion du chargement d'une page par un utilisateur. La relève des courriels dépendrait alors de la présence de quelqu'un dans l'interface : aucun ticket ne serait créé la nuit ni le week-end, et la démonstration du dispositif deviendrait tributaire du hasard le jour de la recette. **Le mode CLI confie le déclenchement au planificateur du système**, seul mode acceptable pour une tâche de fond.

Le mode CLI suppose qu'une tâche planifiée appelle GLPI depuis l'hôte. Elle est posée sur le serveur, et versionnée dans le dépôt sous **serveur/scrciptCron.txt** :

```
# tache planifiee sur l'hote (crontab -e, ou /etc/cron.d/glpi-cron)
* * * * * docker exec glpi_web php /var/www/html/front/cron.php &> /dev/null
```

Une entrée par minute suffit pour l'ensemble des actions automatiques : **cron.php** traite à chaque passage celles dont l'échéance est atteinte. **mailgate** et sa fréquence de cinq minutes ne sont donc servis qu'un passage sur cinq, ce qui est le comportement voulu — la fréquence se règle dans GLPI, pas dans la tâche planifiée.

Deux réserves sur cette ligne de tâche planifiée

Le chemin. `/var/www/html/front/cron.php` ne comporte pas le segment `glpi`, alors que le volume de l'application est monté sur `/var/www/html/glpi` et que la suppression de `install.php` au paragraphe 2.8 porte sur `/var/www/html/glpi/install/install.php`. Les deux chemins ne peuvent pas être corrects en même temps. La figure 20 montre une dernière exécution à 12:02, ce qui est un indice de bon fonctionnement mais pas une preuve : l'action a pu être déclenchée à la main par le bouton *Exécuter*. Trancher en lançant la commande sans redirection (voir les contrôles ci-dessous) : si elle répond *No such file or directory*, corriger en `/var/www/html/glpi/front/cron.php`.

La redirection. `&> /dev/null` est une syntaxe propre à `bash`. Le planificateur exécute la ligne avec `/bin/sh`, qui sur Ubuntu est `dash` et ne la comprend pas de la même façon. Écrire `>/dev/null 2>&1`, forme portable, faute de quoi la sortie et les erreurs ne sont pas maîtrisées.

Ces deux points relèvent du même risque : **une tâche planifiée échoue en silence**. La sortie étant jetée, rien ne signale la panne — sinon l'absence de tickets. Voir l'entrée 12 du registre du chapitre 6.

Contrôles, à exécuter sur le serveur :

```
# 1. la tâche planifiée est bien en place
sudo crontab -l ; ls -l /etc/cron.d/

# 2. execution manuelle SANS redirection : c'est ce qui revele une erreur de chemin
sudo docker exec glpi_web php /var/www/html/front/cron.php

# 3. verifier le chemin reel de l'application dans le conteneur
sudo docker exec glpi_web ls /var/www/html/glpi/front/cron.php
```

Côté interface, les passages sont tracés dans **Configuration > Actions automatiques > mailgate**, onglets *Journaux* et *Statistiques* : c'est là que se lit le nombre de messages traités à chaque relève.

2.10.5 Recette du collecteur

Un message de test a été émis vers la boîte du support, puis le ticket correspondant contrôlé dans GLPI.

Figure 21 – Ticket « Test de création de ticket », ouvert le 2026-09-04 à 11:36:01. Trois éléments valident la chaîne complète : la description reprend le corps du message, la Source de la demande vaut E-Mail, et le demandeur est l'utilisateur JANSEN — l'expéditeur a donc été reconnu, et non traité en anonyme.

Test	Action	Résultat attendu	État
Relève effective	Bouton <i>Exécuter</i> de l'action	La date de dernière exécution s'actualise	Vérifié – fig. 20

	mailgate		
Création du ticket	Émettre un message vers la boîte du support, puis attendre le passage	Un ticket porte l'objet du message en titre et son corps en description	Vérfifié – fig. 21
Origine tracée	Ouvrir le ticket créé	<i>Source de la demande</i> vaut E-Mail	Vérfifié – fig. 21
Demandeur reconnu	Émettre depuis une adresse renseignée sur une fiche utilisateur	Le demandeur est cet utilisateur, et non « anonyme »	Vérfifié – fig. 21
Pièce jointe	Émettre un message porteur d'un PDF	Le document est attaché au ticket	À passer
Réponse à un ticket	Répondre au courriel de notification	Un suivi s'ajoute au ticket existant , sans doublon	À passer
Absence de doublon	Relancer la relève deux fois de suite	Aucun ticket créé au second passage	À passer
Automatisation	Émettre un message et ne rien déclencher à la main	Le ticket apparaît dans les cinq minutes	À passer

Le test « réponse à un ticket » est celui qui distingue un collecteur réellement fonctionnel d'un simple import de messages : il vérifie que GLPI rattache la réponse au ticket d'origine au lieu d'en ouvrir un second. Avec le contrôle d'absence de doublon, il constitue le reste à faire sur ce point — voir l'entrée 13 du registre du chapitre 6.

Le réglage qui évite les doublons

Les champs *Dossier d'archivage des mails acceptés* et *Dossier d'archivage des mails refusés* de la figure 19 sont vides : les messages traités sont donc supprimés de la boîte après relève, et ne sont plus présentés au passage suivant. C'est ce comportement qui empêche les doublons. Le renseigner par un dossier d'archivage est préférable en exploitation — un message qui a échoué reste alors consultable au lieu de disparaître — mais **jamais laisser le collecteur relire indéfiniment les mêmes messages** : chaque passage créerait un ticket de plus.

2.10.6 Notifications sortantes

Le collecteur n'a de sens que couplé aux notifications. Sans elles, le demandeur n'apprend jamais que son ticket a avancé, et surtout il n'a **aucun courriel auquel répondre** : le rattachement des réponses au ticket, testé plus haut, n'a alors pas de déclencheur.

Dans **Configuration > Notifications > Configuration des suivis par courriels** :

Champ	Valeur	Remarque
Activer le suivi par courriels	Oui	
Mode d'envoi	SMTP + SSL, ou SMTP + TLS	Selon le port retenu
Serveur SMTP	Celui du fournisseur de la boîte de support	smtp.gmail.com pour la boîte employée ici
Port	465 en SSL, 587 en TLS	
Adresse d'expédition	L'adresse du compte de support	Celle-là même que le collecteur relève
Identifiant / mot de passe	Ceux du compte dédié	Le même mot de passe d'application

Puis **Configuration > Notifications > Notifications** : activer au minimum « Nouveau ticket » et « Mise à jour d'un ticket ». Un bouton d'envoi de test figure sur la page de configuration et permet de valider le SMTP indépendamment de tout ticket.

La boucle de notification, à traiter avant la mise en service

La boîte relevée par le collecteur est ici **la même que celle qui émet les notifications**. Le scénario à redouter est mécanique : une notification part vers une adresse dotée d'une réponse automatique, cette réponse arrive dans la boîte de support, le collecteur la transforme en ticket, ce ticket déclenche une notification, et la boucle s'entretient. Deux garde-fous : écarter à la relève les messages porteurs d'un en-tête d'auto-réponse, et placer un filtrage des indésirables en amont, faute de quoi chaque message non sollicité devient un ticket. Voir l'entrée 13 du registre du chapitre 6.

2.10.7 Dépannage

Symptôme	Cause probable	Correction
Le menu <i>Collecteurs</i> est absent de GLPI	L'extension PHP imap manque dans l'image du conteneur	<code>sudo docker exec glpi_web php -m grep -i imap</code> ; sans sortie, l'image ne convient pas
Mot de passe refusé par le fournisseur	Mot de passe du compte employé au lieu d'un mot de passe d'application	En générer un dans les paramètres du compte de messagerie
Les messages sont relevés mais aucun ticket n'apparaît	Expéditeur inconnu de GLPI	Renseigner son adresse sur une fiche utilisateur, ou autoriser les tickets anonymes (§2.10.3)
Un ticket est créé à chaque passage pour le même message	Le message n'est ni supprimé ni archivé après relève	Contrôler les dossiers d'archivage du collecteur (§2.10.5)
Rien ne se passe sans déclenchement manuel	Action automatique en mode « GLPI », ou tâche planifiée absente ou en échec	Passer en mode CLI et contrôler la tâche planifiée (§2.10.4)
Aucune notification n'est reçue	SMTP non configuré, ou notifications désactivées	Bouton d'envoi de test de la page de configuration (§2.10.6)

3. Procédure de sécurisation

Ce chapitre applique les quatre couches de défense du paragraphe 1.7. Il est **indissociable du chapitre 2** : à l'issue de l'installation, le service est accessible en clair et sans restriction réseau.

3.1 Certificat TLS

En l'état, les identifiants d'administration transitent en clair sur le réseau : une écoute passive suffirait à les capturer. La correction commence par la production d'un certificat.

```
cd ~/ticketing-projet/nginx
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout certs/glpi.key -out certs/glpi.crt \
  -subj "/C=FR/ST=Loire/L=Nantes/O=MonEntreprise/OU=IT/CN=glpi.local" \
  -addext "subjectAltName=DNS:glpi.local,IP:192.168.1.17,IP:10.8.0.1"
sudo chmod 600 certs/glpi.key
sudo chmod 644 certs/glpi.crt
```

```
ubuntu@ubuntu-server2204:~/ticketing-projet/gmpi$ mkdir -p ~/ticketing-projet/nginx/certs
ubuntu@ubuntu-server2204:~/ticketing-projet/gmpi$ cd ~/ticketing-projet/nginx/
ubuntu@ubuntu-server2204:~/ticketing-projet/nginx$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/gmpi.key -out certs/gmpi.crt \
-subj "/C=FR/ST=Loire/L=Nantes/O=MonEntreprise/OU=IT/CN=gmpi.local"
```

Figure 11 – Génération du certificat auto-signé, RSA 2048 bits, valable 365 jours. La capture est recadrée : le reste de la sortie n'est que l'indicateur de progression d'OpenSSL.

L'option ``-addext`` est un ajout à la commande d'origine

Elle ne figurait pas dans la commande exécutée lors du déploiement initial, et son absence explique une partie de l'avertissement affiché par le navigateur : les navigateurs modernes ignorent le champ **CN** et ne se fient qu'au

subjectAltName. Sans elle, l'accès par adresse IP déclenche une erreur de non-correspondance de nom **en plus** de celle d'autorité inconnue. La conserver lors de tout renouvellement (paragraphe 4.5).

3.2 Reverse proxy NGINX

Créer `~/ticketing-projet/nginx/nginx.conf` avec le contenu de l'**annexe B**. Le fichier est monté sur `/etc/nginx/conf.d/default.conf` dans le conteneur : il remplace donc le site par défaut de NGINX. Il déclare deux blocs **server** :

- le premier écoute en 80 et renvoie un **301 vers HTTPS** : aucun accès en clair ne subsiste, même si un utilisateur saisit l'adresse sans le préfixe sécurisé ;
- le second termine le TLS, restreint les protocoles à **TLS 1.2 et 1.3** – excluant donc SSLv3, TLS 1.0 et 1.1, tous obsolètes – et les suites cryptographiques à **HIGH:!aNULL:!MD5** ;
- le bloc **location** / relaie vers `http://glpi:80` en résolvant le conteneur par son nom de service, et transmet **X-Real-IP**, **X-Forwarded-For** et **X-Forwarded-Proto** pour que GLPI journalise l'adresse réelle du client et se sache derrière un proxy HTTPS.

```
ubuntu@ubuntuserver2204:~/ticketing-projet/nginx$ nano nginx.conf
ubuntu@ubuntuserver2204:~/ticketing-projet/nginx$ cat nginx.conf
server {
    listen 80;
    server_name _;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    ssl_certificate /etc/nginx/certs/glpi.crt;
    ssl_certificate_key /etc/nginx/certs/glpi.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://glpi:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Figure 12 – Configuration du reverse proxy.

Ajouter ensuite le service **nginx** au fichier de composition, sous le nom de conteneur **glpi_proxy**. **Le point déterminant est ce qui disparaît** : la section **ports** du service **glpi** est supprimée. Sans cette suppression, le port 80 du conteneur GLPI resterait publié et offrirait un accès en clair contournant entièrement le proxy et son chiffrement.

```

ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ nano docker-compose.yml
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ cat docker-compose.yml
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

  nginx:
    image: nginx:latest
    container_name: glpi_proxy
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ../nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ../nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - glpi

volumes:
  db_data:
  glpi_data:
ubuntu@ubuntuserver2204:~/ticketing-projet/glpi$ █

```

Figure 13 – Fichier de composition final. Le service `glpi` ne comporte plus de section `ports` : comparer avec la figure 5.

Élément du fichier	Pourquoi il est ainsi
Le service <code>glpi</code> n'a aucune section <code>ports</code>	Un port publié sur <code>glpi</code> offrirait un accès HTTP en clair contournant NGINX. C'est la principale erreur à ne pas réintroduire.
Le service <code>db</code> n'a aucune section <code>ports</code>	MariaDB ne doit être joignable que depuis le réseau des conteneurs, jamais depuis l'hôte ni le réseau local.
<code>nginx</code> monte ses fichiers en <code>:ro</code>	Lecture seule : un conteneur compromis ne peut pas réécrire la configuration du proxy ni remplacer le certificat.

3.3 Bascule en HTTPS et contrôles

```
cd ~/ticketing-projet/glpi
sudo docker compose down
sudo docker compose up -d
```

```
ubuntu@ubuntu-server2204:~/ticketing-projet/glpi$ sudo docker compose down
WARN[0000] /home/ubuntu/ticketing-projet/glpi/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Container glpi_proxy   Removed
  ✓ Container glpi_web     Removed
  ✓ Container glpi_db      Removed
  ✓ Network glpi_default   Removed
ubuntu@ubuntu-server2204:~/ticketing-projet/glpi$ sudo docker compose up -d
WARN[0000] /home/ubuntu/ticketing-projet/glpi/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
  ✓ Network glpi_default   Created
  ✓ Container glpi_db      Started
  ✓ Container glpi_web     Started
  ✓ Container glpi_proxy   Started
ubuntu@ubuntu-server2204:~/ticketing-projet/glpi$
```

Figure 14 – Résultat attendu : quatre éléments démarrés – le réseau et les trois conteneurs.

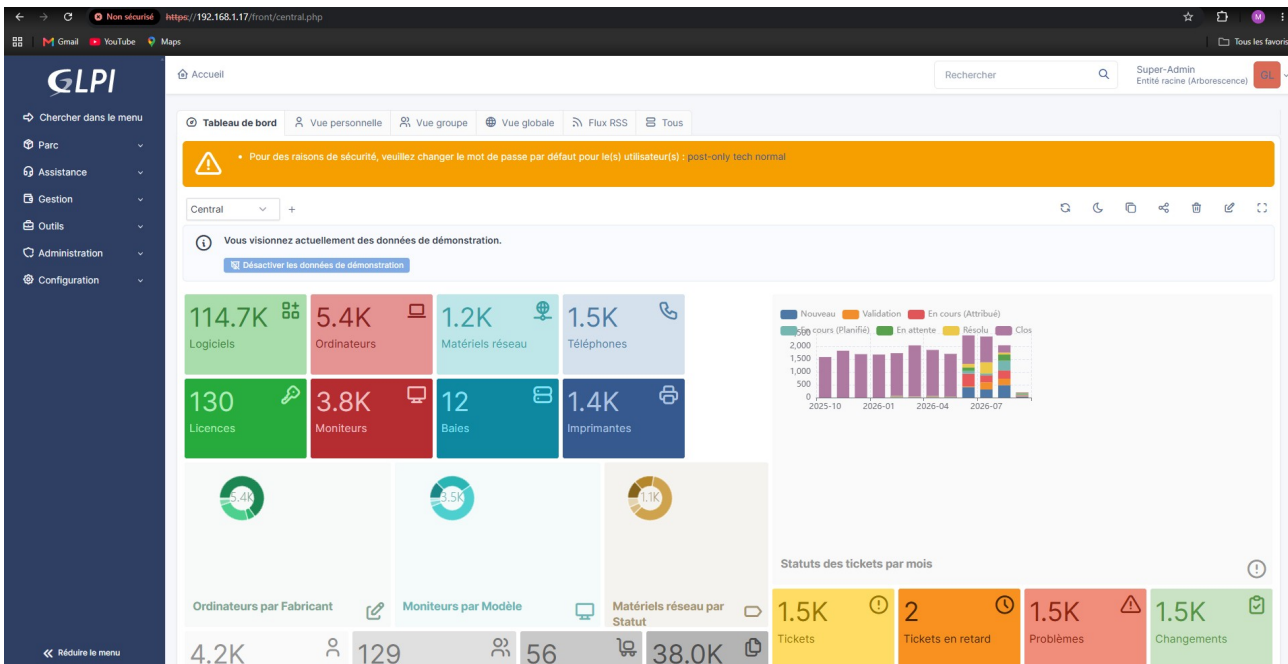


Figure 15 – GLPI répond en HTTPS. L'avertissement du navigateur est attendu : le certificat est auto-signé.

L'avertissement « Non sécurisé » subsiste et doit être interprété correctement : le **chiffrement du transport est bien effectif**, seule l'**authentification du serveur** n'est pas vérifiable, faute de signature par une autorité reconnue. Sa levée passe par une autorité de certification interne – voir l'entrée 5 du registre du chapitre 6.

Contrôles à effectuer dans l'ordre :

```
# 1. les trois conteneurs sont Up, et seul glpi_proxy publie 80 et 443
sudo docker compose ps

# 2. la configuration NGINX est syntaxiquement valide
sudo docker exec glpi_proxy nginx -t

# 3. le port 80 renvoie bien une redirection 301
curl -I http://localhost/

# 4. le port 443 répond (-k : on accepte le certificat auto-signé)
curl -Ik https://localhost/

# 5. le certificat présente les bons noms et la bonne échéance
echo | openssl s_client -connect localhost:443 2>/dev/null \
| openssl x509 -noout -subject -ext subjectAltName -enddate
```


3.4 Tunnel OpenVPN

C'est la couche qui applique la consigne centrale du cahier de recettes. Plutôt qu'exposer GLPI sur Internet, le service devient joignable uniquement depuis l'intérieur d'un tunnel chiffré.

3.4.1 Installation du serveur

L'installation s'appuie sur le script de déploiement OpenVPN d'angristan, largement utilisé, qui automatise la mise en place de la PKI.

```
cd ~
curl -O https://raw.githubusercontent.com/angristan/openvpn-install/master/openvpn-install.sh
chmod +x openvpn-install.sh
sudo ./openvpn-install.sh install
```

```
ubuntu@ubuntu2204:~$ curl -O https://raw.githubusercontent.com/angristan/openvpn-install/master/openvpn-install.sh
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 170k 100 170k    0     0  321k      0  0 --:--:-- --:--:-- --:--:-- 321k
ubuntu@ubuntu2204:~$ chmod +x openvpn-install.sh
ubuntu@ubuntu2204:~$ sudo ./openvpn-install.sh
OpenVPN installer and manager

Usage: openvpn-install <command> [options]

Commands:
install      Install and configure OpenVPN server
uninstall    Remove OpenVPN server
client       Manage client certificates
server       Server management
interactive   Launch interactive menu

Global Options:
--verbose    Show detailed output
--log <path> Log file path (default: openvpn-install.log)
--no-log     Disable file logging
--no-color   Disable colored output
-h, --help   Show help

Run 'openvpn-install <command> --help' for command-specific help.
ubuntu@ubuntu2204:~$ sudo ./openvpn-install.sh install
[INFO] === OpenVPN Non-Interactive Install ===
[INFO] Running in non-interactive mode with the following settings:
[INFO]   ENDPOINT=37.65.173.112
[INFO]   ENDPOINT_TYPE=4
[INFO]   CLIENT_IPV4=y
[INFO]   CLIENT_IPV6=n
[INFO]   VPN_SUBNET_IPV4=10.8.0.0
[INFO]   VPN_SUBNET_IPV6=fd42:42:42:42::
[INFO]   ROUTE_INTERNET=y
[INFO]   CLIENT_TO_CLIENT=n
[INFO]   LOCAL_NETWORKS=none
[INFO]   PORT=1194
[INFO]   PROTOCOL=udp
[INFO]   DNS=cloudflare
[INFO]   MULTI_CLIENT=n
[INFO]   AUTH_MODE=pki
[INFO]   CLIENT=client
[INFO]   CLIENT_CERT_DURATION_DAYS=3650
[INFO]   SERVER_CERT_DURATION_DAYS=3650
[INFO] Setting up official OpenVPN repository...
> apt-get update
> apt-get install -y ca-certificates curl
> mkdir -p /etc/apt/keyrings
> curl -fsSL https://swupdate.openvpn.net/repos/repo-public.gpg -o /etc/apt/keyrings/openvpn-repo-public.asc
[INFO] Updating package lists with new repository...
> apt-get update
[INFO] OpenVPN official repository configured
[INFO] Installing OpenVPN and dependencies...
> apt-get install -y openvpn iptables openssl curl ca-certificates tar dnsutils socat
```

Figure 16 – Installation en mode non interactif. Le script affiche les paramètres retenus avant de configurer le dépôt officiel OpenVPN et d'installer les dépendances.

Paramètres appliqués lors du déploiement initial :

Paramètre	Valeur	Signification
ENDPOINT / ENDPOINT_TYPE	37.65.173.112 / 4	Adresse IPv4 publique annoncée aux clients
PORT / PROTOCOL	1194 / udp	Écoute du serveur ; UDP réduit la latence du tunnel
VPN_SUBNET_IPV4	10.8.0.0/24	Plage attribuée aux clients ; le serveur prend 10.8.0.1
AUTH_MODE	pki	Authentification par certificat client, et non par mot de passe partagé

CLIENT_TO_CLIENT	n	Bon réglage : les clients ne peuvent pas se joindre entre eux, ce qui limite les déplacements latéraux si un poste est compromis
ROUTE_INTERNET	y	L'intégralité du trafic client passe par le tunnel
DNS	cloudflare	Résolution via le tunnel, sans fuite de requêtes
CLIENT_IPV6	n	IPv6 désactivé côté client, ce qui évite un contournement du filtrage IPv4
SERVER_CERT_DURATION_DAYS	3650	À réduire : dix ans de validité
CLIENT_CERT_DURATION_DAYS	3650	À réduire : dix ans de validité

3.4.2 Créer un accès client

Le script gère les certificats clients par sa sous-commande **client**. La syntaxe exacte des options dépend de la version téléchargée : la vérifier avant usage, ou passer par le menu interactif qui reste le chemin le plus sûr.

```
cd ~
sudo ./openvpn-install.sh client --help    # syntaxe exacte de la version installée
sudo ./openvpn-install.sh interactive      # menu : ajouter / révoquer un client
```

Le profil **.ovpn** généré contient la clé et le certificat du client : **c'est un secret complet d'accès au réseau**. Il est écrit dans le répertoire personnel de **root**. Transmission puis nettoyage :

```
# sur le serveur : rendre le profil récupérable par l'utilisateur ubuntu
sudo cp /root/poste-technicien.ovpn /home/ubuntu/
sudo chown ubuntu:ubuntu /home/ubuntu/poste-technicien.ovpn

# depuis le poste client : récupération par un canal chiffré
scp ubuntu@192.168.1.17:/home/ubuntu/poste-technicien.ovpn .

# sur le serveur : effacer la copie intermédiaire
shred -u /home/ubuntu/poste-technicien.ovpn
```

Un profil par personne, jamais de profil partagé

Émettre un profil distinct par utilisateur est ce qui rend la révocation possible : retirer l'accès à une personne sans reconfigurer tout le monde. Un profil partagé entre plusieurs techniciens interdit toute révocation ciblée et rend les journaux de connexion inexploitable.

3.4.3 Connexion et révocation

```
# poste client Linux
sudo apt install openvpn -y
sudo openvpn --config poste-technicien.ovpn

# vérification, une fois le tunnel monte
ip addr show tun0
ping -c 3 10.8.0.1

# révocation d'un accès, sur le serveur
sudo ./openvpn-install.sh interactive    # puis : révoquer un client
```

3.4.4 Variante : mise en place manuelle de la PKI avec Easy-RSA

L'installation décrite au paragraphe 3.4.1 s'appuie sur un script qui automatise la création de l'autorité de certification et l'écriture de la configuration. La procédure manuelle équivalente est consignée ici pour deux

raisons : elle permet de reconstruire le tunnel **sans dépendre d'un script tiers**, et elle rend visibles les objets cryptographiques que le script produit sans les nommer — ce qu'il faut savoir expliquer.

Une seule PKI à la fois

Cette variante **remplace** le paragraphe 3.4.1, elle ne s'y ajoute pas. Dérouler les deux sur la même machine produirait deux autorités de certification concurrentes et un service `openvpn-server@server` dont la configuration ne correspondrait plus aux certificats distribués aux clients — panne dont la cause est difficile à isoler, le service démarrant normalement.

Étape 1 — paquets.

```
sudo apt update && sudo apt install openvpn easy-rsa -y
```

Étape 2 — autorité de certification et identité du serveur. `make-cadir` crée un répertoire de travail contenant sa propre copie des scripts Easy-RSA : la PKI est ainsi isolée des mises à jour du paquet.

```
make-cadir ~/openvpn-ca && cd ~/openvpn-ca
./easyrsa init-pki
./easyrsa build-ca nopass
./easyrsa gen-req server nopass
./easyrsa sign-req server server
./easyrsa gen-dh
openvpn --genkey secret ta.key
```

Les fichiers produits n'ont pas la même sensibilité, et c'est ce qui commande leur traitement :

Fichier	Rôle	Conséquence d'une divulgation
<code>pki/ca.crt</code>	Certificat de l'autorité ; distribué à tous, il permet à chacun de vérifier les autres	Aucune : c'est une donnée publique, elle figure dans chaque profil client
<code>pki/private/ca.key</code>	Clé de l'autorité ; signe les certificats	Compromission totale : un tiers peut émettre des accès valides que le serveur acceptera. À conserver hors de la machine dès que la PKI a une durée de vie réelle
<code>pki/issued/server.crt</code> , <code>pki/private/server.key</code>	Identité du serveur	Usurpation du concentrateur : un faux serveur peut se faire passer pour le vrai
<code>pki/dh.pem</code>	Paramètres Diffie-Hellman de l'échange de clés	Aucune : donnée publique
<code>ta.key</code>	Clé de la couche <code>tls-crypt</code> , qui chiffre la négociation TLS elle-même	Le port 1194 redevient visible d'un balayage et le service exposé aux tentatives de connexion

`build-ca nopass` produit une autorité sans phrase de passe

Commode en laboratoire — aucune saisie à chaque émission de certificat — mais la clé de l'autorité est alors exploitable par quiconque lit le fichier. Sur une PKI destinée à durer, omettre `nopass` et protéger `ca.key` par une phrase de passe, l'autorité n'étant sollicitée qu'à la création ou à la révocation d'un accès.

Étape 3 — certificat du client. Un jeu par personne, jamais un jeu partagé.

```
cd ~/openvpn-ca
./easyrsa gen-req client1 nopass
./easyrsa sign-req client client1
```

Étape 4 — configuration du serveur. Les fichiers nécessaires au service sont recopiés dans son répertoire ; la PKI, elle, reste dans `~/openvpn-ca` où se feront les émissions et révocations ultérieures.

```
sudo cp pki/ca.crt pki/issued/server.crt pki/private/server.key pki/dh.pem ta.key \
/etc/openvpn/server/
```

Puis `/etc/openvpn/server/server.conf` :

```
port 1194
proto udp
dev tun
ca ca.crt
cert server.crt
key server.key
dh dh.pem
tls-crypt ta.key
server 10.8.0.0 255.255.255.0
push "redirect-gateway def1 bypass-dhcp"
push "dhcp-option DNS 1.1.1.1"
keepalive 10 120
data-ciphers AES-256-GCM
cipher AES-256-GCM
user nobody
group nogroup
persist-key
persist-tun
crl-verify crl.pem      # etape 8 : sans cette ligne, une revocation est sans effet
status openvpn-status.log
verb 3
```

Directive	Effet, et lien avec le reste du dispositif	Renvoi
<code>server 10.8.0.0 255.255.255.0</code>	Plage attribuée aux clients ; le serveur prend 10.8.0.1, adresse par laquelle GLPI est joint	\$1.3, \$1.5
<code>push "redirect-gateway def1 bypass-dhcp"</code>	Tout le trafic du client passe par le tunnel — équivaut au paramètre <code>ROUTE_INTERNET</code> du script	\$3.4.1
<code>tls-crypt ta.key</code>	Chiffre la négociation TLS : un balayage du port 1194 n'obtient aucune réponse identifiable	\$3.8
Absence de <code>client-to-client</code>	Volontaire : les clients du tunnel ne peuvent pas se joindre entre eux, ce qui limite les déplacements latéraux si un poste est compromis	\$3.4.1
<code>user nobody</code> / <code>group nogroup</code>	Le démon abandonne ses privilèges après création de l'interface <code>tun0</code>	
<code>data-ciphers AES-256-GCM</code>	Directive en vigueur depuis OpenVPN 2.5 ; <code>cipher</code> est conservé pour les clients plus anciens et sera à retirer à terme	
<code>crl-verify crl.pem</code>	Fait consulter la liste de révocation à chaque connexion	Étape 8

Étape 5 — routage et pare-feu. Le serveur doit relayer les paquets du tunnel vers les conteneurs, ce que le noyau refuse par défaut.

```
sudo sysctl -w net.ipv4.ip_forward=1
sudo sed -i 's|^#\?net.ipv4.ip_forward=.*|net.ipv4.ip_forward=1|' /etc/sysctl.conf
sudo sysctl -p
sudo ufw allow 1194/udp
```

`sysctl -w` ne survit pas au redémarrage

D'où la seconde commande, qui inscrit le réglage dans `/etc/sysctl.conf`. C'est exactement la même catégorie d'erreur que la règle `iptables` non persistante du paragraphe 3.6.1 : **une mesure appliquée à chaud, correcte au moment du contrôle, absente après le premier redémarrage**. Les deux se vérifient de la même façon — redémarrer, puis relire l'état.

Étape 6 — démarrage et contrôles.

```
sudo systemctl enable --now openvpn-server@server
sudo systemctl status openvpn-server@server --no-pager
sudo journalctl -u openvpn-server@server -n 30
ip -br a show tun0      # l'interface doit exister et porter 10.8.0.1
sudo ss -ulnp | grep 1194 # le service écoute bien en UDP
```

Étape 7 — profil du client. Le profil `.ovpn` rassemble la configuration et les certificats en un fichier unique, ce qui évite de distribuer quatre fichiers séparés.

```

client
dev tun
proto udp
remote 37.65.173.112 1194
resolv-retry infinite
nobind
persist-key
persist-tun
remote-cert-tls server
data-ciphers AES-256-GCM
cipher AES-256-GCM
verb 3

<ca>          contenu de pki/ca.crt          </ca>
<cert>        contenu de pki/issued/client1.crt </cert>
<key>         contenu de pki/private/client1.key </key>
<tls-crypt>   contenu de ta.key              </tls-crypt>

```

remote-cert-tls server n'est pas facultatif : cette directive fait refuser au client tout serveur dont le certificat ne porte pas l'usage « serveur ». Sans elle, **un porteur de certificat client émis par la même autorité pourrait se faire passer pour le concentrateur** et détourner les connexions des autres postes.

Le fichier .ovpn est un secret complet d'accès au réseau

Il contient la clé privée du client : quiconque l'obtient entre dans le tunnel et atteint GLPI. Le transmettre par un canal chiffré, un profil par personne, et effacer les copies intermédiaires — les commandes figurent au paragraphe 3.4.2, qui s'applique intégralement à cette variante.

```

# poste client
sudo apt install openvpn -y
sudo openvpn --config client1.ovpn

# controles, une fois le tunnel monte
ip addr show tun0
ping -c 3 10.8.0.1
curl -Ik --max-time 5 https://10.8.0.1/      # GLPI doit repondre

```

Étape 8 — révocation d'un accès. C'est l'opération qui donne sa valeur au choix « un profil par personne ».

```

cd ~/openvpn-ca
./easysrsa revoke client1
./easysrsa gen-crl
sudo cp pki/crl.pem /etc/openvpn/server/
sudo systemctl restart openvpn-server@server

```

Une révocation sans crl-verify est purement décorative

Révoquer un certificat inscrit son numéro de série dans la liste de révocation de la PKI. Mais **le serveur ne consulte cette liste que si `server.conf` déclare `crl-verify crl.pem`** — omission classique. Sans cette ligne, le certificat révoqué continue d'être accepté, et l'accès qu'on croit avoir retiré reste ouvert. Contrôler après toute révocation que la connexion avec l'ancien profil est bien refusée : c'est le seul test qui atteste du retrait.

3.5 Pare-feu UFW

Le tunnel n'a de valeur que si le service n'est pas également joignable par un autre chemin. Le pare-feu est configuré en refus par défaut, puis ouvert au strict nécessaire.

```

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw allow 1194/udp
sudo ufw allow in on tun0
sudo ufw enable
sudo ufw status verbose

```

```

ubuntu@ubuntu-server2204:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server2204:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
ubuntu@ubuntu-server2204:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
ubuntu@ubuntu-server2204:~$ sudo ufw allow 1194/udp
Rules updated
Rules updated (v6)
ubuntu@ubuntu-server2204:~$ sudo ufw allow in on tun0
Rules updated
Rules updated (v6)
ubuntu@ubuntu-server2204:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
ubuntu@ubuntu-server2204:~$ █

```

Figure 17 – Mise en place du jeu de règles. La demande de confirmation de `ufw enable` avertit que l'opération peut interrompre les connexions SSH existantes.

Ordre des commandes

Ouvrir 22/tcp **avant** d'activer le pare-feu. L'inverse coupe la session SSH en cours et rend le serveur inaccessible s'il n'y a pas d'accès console. C'est précisément ce que signale l'avertissement affiché par `ufw enable`.

Règle	Justification
<code>default deny incoming</code>	Tout ce qui n'est pas explicitement autorisé est refusé : c'est la posture correcte, l'inverse d'une liste noire toujours incomplète
<code>default allow outgoing</code>	Le serveur doit pouvoir sortir pour ses mises à jour et, pour interroger en 993/tcp la boîte de courriels du collecteur (paragraphe 2.10)
<code>allow 22/tcp</code>	Administration SSH ; seule ouverture directe conservée
<code>allow 1194/udp</code>	Point d'entrée du tunnel OpenVPN
<code>allow in on tun0</code>	Tout ce qui arrive par le tunnel est accepté. C'est cette règle qui rend GLPI accessible aux clients VPN
443/tcp non ouvert	Omission délibérée, cœur de la consigne : HTTPS n'est pas joignable depuis l'extérieur du tunnel

État attendu de `ufw status verbose` :

```

Status: active
Default: deny (incoming), allow (outgoing), disabled (routed)

To Action From
--
22/tcp ALLOW IN Anywhere
1194/udp ALLOW IN Anywhere
Anywhere on tun0 ALLOW IN Anywhere

```

3.6 Isolation des conteneurs : la chaîne DOCKER-USER

Le problème. Le jeu de règles précédent ne suffit pas. Docker publie les ports de ses conteneurs en écrivant directement dans `iptables` : une règle de traduction d'adresse dans la table `nat`, et une autorisation de transit dans la chaîne `FORWARD`. Ces règles sont évaluées **avant** celles que gère UFW, lequel travaille sur

INPUT. Conséquence : **un port publié par un conteneur reste joignable malgré un pare-feu en `deny incoming`**. Le pare-feu n'est pas contourné par malveillance, il n'est simplement jamais consulté pour ce trafic, qui est routé et non destiné à l'hôte. C'est une cause récurrente d'exposition involontaire de services conteneurisés.

La solution. Docker réserve la chaîne **DOCKER-USER** à l'administrateur : il la traverse avant d'appliquer ses propres règles et ne la réécrit jamais au redémarrage du démon.

```
sudo iptables -I DOCKER-USER -i enp0s8 -j DROP
```

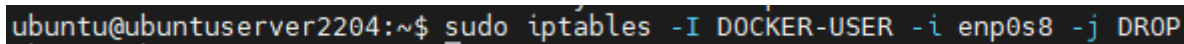


Figure 18 – Blocage de tout trafic à destination des conteneurs arrivant par l'interface physique.

Le filtre porte sur l'**interface d'entrée**, ce qui sépare exactement les deux chemins d'accès :

Trafic	Interface d'entrée	Résultat
Poste du réseau local vers l'interface web de GLPI	enp0s8	Rejeté par la règle DOCKER-USER
Client connecté au tunnel VPN vers l'interface web de GLPI	tun0	Accepté : la règle ne s'applique pas, le paquet atteint NGINX

C'est cette règle, et non le pare-feu seul, qui rend la consigne « n'autoriser que le tunnel du VPN en entrée » réellement effective.

```
# verification : la regle doit apparaitre en position 1
sudo iptables -L DOCKER-USER -n -v --line-numbers
```

3.6.1 Rendre la règle persistante — correctif prioritaire

Une unité **systemd** est préférable à **iptables-persistent** dans ce cas précis : elle garantit que la règle est posée **après** le démarrage du démon Docker, donc après que celui-ci a recréé la chaîne. Avec **iptables-persistent** seul, la restauration peut intervenir avant Docker et être effacée par lui.

```
sudo tee /etc/systemd/system/docker-isolation.service > /dev/null <<'EOF'
[Unit]
Description=Isolation des conteneurs Docker vis-a-vis du reseau local
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
ExecStart=/usr/sbin/iptables -I DOCKER-USER -i enp0s8 -j DROP
ExecStop=/usr/sbin/iptables -D DOCKER-USER -i enp0s8 -j DROP

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable --now docker-isolation.service
sudo systemctl status docker-isolation.service
```

Recette du correctif, à exécuter pour valider qu'il fonctionne réellement :

```
# 1. redemarrer le serveur
sudo reboot

# 2. au retour, la regle doit etre presente sans intervention
sudo iptables -L DOCKER-USER -n --line-numbers

# 3. depuis un poste du reseau local, HORS tunnel : doit ECHOUER
curl -Ik --max-time 5 https://192.168.1.17/

# 4. depuis un poste avec le tunnel monte : doit REPONDRE
curl -Ik --max-time 5 https://10.8.0.1/
```

Les étapes 3 et 4 sont le seul contrôle qui atteste vraiment de l'isolation. **Lire le jeu de règles ne suffit pas** : c'est exactement l'erreur qui laisse un service exposé malgré un pare-feu correctement configuré. Toute intervention touchant au réseau, au pare-feu ou à Docker doit se terminer par ces deux commandes.

3.7 Durcissement de GLPI

Mesure	Emplacement dans GLPI	État actuel
Mot de passe du compte glpi	Administration > Utilisateurs	Fait
Mots de passe de tech , normal , post-only	Administration > Utilisateurs	À faire – identifiants par défaut publics
Désactivation des données de démonstration	Tableau de bord, bouton dédié	À faire – 5 400 ordinateurs et 1 500 tickets fictifs
Double authentification TOTP	Configuration > Authentification	À faire
Politique de mot de passe (longueur, complexité, expiration)	Configuration > Générale > Sécurité	À faire
Suppression de install/install.php	Système de fichiers du conteneur	À vérifier – et à refaire après chaque mise à jour d'image
Journalisation des connexions	Administration > Journaux	Actif par défaut

3.8 Détection et prévention d'intrusion : Suricata

Cinquième couche du dispositif du paragraphe 1.7. Les quatre premières empêchent d'entrer ; celle-ci **constate ce qui est tenté** et, en mode prévention, l'interrompt. Le cahier de recettes la demande en mode IPS et signale d'emblée le point délicat : « attention aux fichiers de configuration ».

État : procédure établie, déploiement à réaliser

Suricata n'est pas installé sur le serveur à la date de rédaction. Ce paragraphe est donc **la procédure à appliquer, non le compte rendu d'une installation** : à la différence du reste des chapitres 2 et 3, il n'est pas illustré de captures relevées sur la machine. La liste des captures à produire lors du déploiement figure au paragraphe 3.8.6, et l'écart correspondant est l'entrée 14 du registre du chapitre 6.

3.8.1 Où placer la sonde : la vraie difficulté

La remarque du cahier de recettes vise les fichiers de configuration, mais dans cette architecture la difficulté première est ailleurs : **choisir l'interface d'écoute**. Entre le tunnel, le TLS et les conteneurs, le même flux est lisible ou opaque selon l'endroit où l'on se place.

Interface	Ce que Suricata y voit	Ce que cela permet de détecter
<code>enp0s8</code> — réseau local	Le tunnel OpenVPN, chiffré — et rien d'autre, le pare-feu ayant fermé le reste	Balayages de ports, sollicitations sur 1194/udp et 22/tcp, trafic non sollicité
<code>tun0</code> — tunnel	Le trafic du VPN déchiffré, mais le HTTPS vers NGINX y reste chiffré en TLS	Force brute SSH, métadonnées TLS : nom de serveur demandé, empreinte du client
<code>br-xxxxxxxxxxxx</code> — réseau <code>glpi_default</code>	Le HTTP en clair entre <code>glpi_proxy</code> et <code>glpi_web</code>	Injections SQL, accès à <code>/install</code> , force brute sur le formulaire de connexion : toutes les règles applicatives

C'est le troisième point d'écoute qui rend le dispositif utile. **NGINX termine le TLS et relaie du HTTP en clair vers le conteneur GLPI**, sur le réseau `glpi_default` : une sonde posée là voit les URL complètes sans avoir à déchiffrer quoi que ce soit. La version précédente de ce document annonçait la difficulté — « le chiffrement aveugle la sonde » — sans la trancher ; c'est la réponse. Une sonde posée uniquement sur `enp0s8`, configuration la plus fréquente parce que la plus intuitive, ne verrait que des paquets OpenVPN opaques et donnerait une couverture illusoire.

Suricata sait écouter plusieurs interfaces simultanément : **les trois sont retenues**, aucune ne donnant à elle seule une vue complète.

3.8.2 Relever les interfaces et le sous-réseau Docker

Aucune de ces valeurs ne peut être recopiée d'une documentation : le nom de l'interface du pont Docker dérive de l'identifiant du réseau, propre à chaque installation.

```
ip -br a                                # enp0s8, tun0, docker0, br-xxxxxxxxxxxx
docker network ls

# nom de l'interface du reseau du projet :
# br- suivi des 12 premiers caracteres de l'identifiant du reseau
docker network inspect glpi_default -f '{{.Id}}' | cut -c1-12

# sous-reseau attribue a ce reseau (typiquement 172.18.0.0/16)
docker network inspect glpi_default -f '{{range .IPAM.Config}}{{.Subnet}}{{end}}'
```

Noter les deux valeurs : le nom `br-...` servira au bloc `af-packet`, le sous-réseau au bloc `HOME_NET`. Le réseau du projet s'appelle `glpi_default`, du nom du répertoire contenant le fichier de composition — même règle de préfixage que pour les volumes, voir l'encadré du paragraphe 1.4.

3.8.3 Installation

```
sudo add-apt-repository -y ppa:oisf/suricata-stable
sudo apt update && sudo apt install -y suricata jq
suricata --build-info | head -20

sudo systemctl stop suricata      # on configure AVANT de démarrer
```

Le dépôt de l'OISF livre Suricata 7, là où les paquets d'Ubuntu 22.04 restent sur une version antérieure. `jq` sert à exploiter le journal structuré `eve.json` au paragraphe 3.8.6. Le service est arrêté d'emblée : démarrer sur la configuration livrée par défaut n'apporte rien et masque les erreurs qu'on cherche à voir.

3.8.4 Configuration : les trois blocs qui comptent

Tout se joue dans `/etc/suricata/suricata.yaml`. Trois blocs seulement demandent une intervention, mais aucun ne peut être laissé en l'état.

Variables réseau. Elles déterminent ce que Suricata considère comme interne.

```
vars:
  address-groups:
    HOME_NET: "[192.168.1.0/24,10.8.0.0/24,172.18.0.0/16]"
    EXTERNAL_NET: "!$HOME_NET"
  port-groups:
    HTTP_PORTS: "[80,443]"
```

Les trois sous-réseaux réellement en jeu : le réseau local, le tunnel, et le réseau des conteneurs — dont la valeur est celle relevée au paragraphe 3.8.2. **Si le sous-réseau Docker manque**, le trafic entre `glpi_proxy` et `glpi_web` est classé comme externe et les règles applicatives ne se déclenchent pas comme prévu : panne d'analyse sans message d'erreur.

Interfaces écoutées.

```
af-packet:
  - interface: enp0s8
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes
    use-mmap: yes
  - interface: tun0
    cluster-id: 98
    cluster-type: cluster_flow
    defrag: yes
  - interface: br-xxxxxxxxxxxx # nom relève au 3.8.2
    cluster-id: 97
    cluster-type: cluster_flow
    defrag: yes
```

Le piège annoncé par le cahier de recettes

Le fichier livré par défaut déclare `interface: eth0`. **Aucune interface ne porte ce nom sur ce serveur.** Suricata démarre pourtant sans erreur, écrit ses journaux, et n'analyse rien. C'est une panne silencieuse de la pire espèce : le service est *active (running)*, les fichiers de journal existent et se remplissent d'entrées de démarrage, et la sonde est aveugle. Seul le test de configuration du paragraphe 3.8.5 la révèle.

Deuxième détail à ne pas manquer : chaque `cluster-id` doit être **distinct** d'une interface à l'autre. Deux interfaces partageant le même identifiant se disputent la même grappe de capture, avec des pertes de paquets à la clé.

Fichiers de règles.

```
default-rule-path: /var/lib/suricata/rules
rule-files:
  - suricata.rules
  - local.rules
```

3.8.5 Valider avant de démarrer

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
# attendu : "Configuration provided was successfully loaded"
```

Cette commande est la réponse directe à la remarque du cahier de recettes. Elle contrôle la syntaxe du fichier, **l'existence effective des interfaces déclarées** et la validité de chaque règle chargée. Elle se relance après toute modification, avant tout redémarrage du service. Son échec est bloquant : un **systemctl restart** sur une configuration invalide laisse le service dans un état qui, vu de **systemctl status**, ressemble au bon.

3.8.6 Jeux de règles et recette de la détection

Deux sources complémentaires : Emerging Threats Open pour la couverture générale, un fichier local pour ce que cette infrastructure expose précisément.

```
sudo suricata-update update-sources
sudo suricata-update enable-source et/open
sudo suricata-update
```

Les règles propres au lab sont écrites dans **/var/lib/suricata/rules/local.rules**, avec des numéros pris dans la plage réservée à cet usage — à partir de 1 000 000, pour ne jamais entrer en collision avec un jeu de règles téléchargé. Le contenu du fichier figure à l'annexe E ; il couvre cinq points d'exposition.

SID	Ce que la règle détecte	Interface où elle se déclenche
1000001	Force brute SSH : cinq ouvertures de session en soixante secondes depuis une même source	enp0s8, tun0
1000002	Balayage du port OpenVPN : dix paquets vers 1194/udp en soixante secondes	enp0s8
1000003	Injection SQL suspectée : union puis select dans une URL	br-... (HTTP en clair)
1000004	Accès au répertoire d'installation /install/	br-...
1000005	Force brute sur /front/login.php : dix requêtes POST en soixante secondes	br-...

Les trois dernières ne se déclenchent **que grâce à l'écoute du réseau Docker** : sur **enp0s8** ou **tun0**, l'URL est chiffrée et le motif introuvable. C'est la démonstration concrète du choix du paragraphe 3.8.1, et la réponse à faire si l'on demande pourquoi trois interfaces plutôt qu'une.

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
sudo systemctl enable --now suricata
sudo systemctl status suricata --no-pager
```

Sollicitations volontaires, depuis le poste client et tunnel monté. Elles servent de repère horaire autant que de test :

```
# déclenchement de la règle 1000003
curl -k "https://10.8.0.1/?id=1%20union%20select%201"

# déclenchement de la règle 1000004
curl -k "https://10.8.0.1/install/install.php"
```

Puis, sur le serveur :

```
sudo tail -n 20 /var/log/suricata/fast.log

sudo jq 'select(.event_type=="alert") | {heure:.timestamp, sig:.alert.signature,
  src:.src_ip, uri:.http.url}' /var/log/suricata/eve.json | tail -30

sudo suricatasc -c "iface-stat enp0s8" # compteurs de paquets non nuls
```

Le contrôle porte sur **trois éléments simultanés** : le bon SID, l'adresse source du poste client, et un horodatage correspondant au **curl**. Une alerte au bon SID mais dont l'horodatage est sans rapport ne

prouve rien — elle peut provenir d'un autre trafic. Si `fast.log` reste vide, la cause est presque toujours l'interface : reprendre le paragraphe 3.8.4.

Test	Action	Résultat attendu
Configuration valide	<code>suricata -T -c /etc/suricata/suricata.yaml -v</code>	« Configuration provided was successfully loaded »
Les interfaces sont réellement écoutées	<code>sudo suricatasc -c "iface-stat enp0s8"</code>	Compteurs de paquets non nuls, et croissants entre deux appels
Détection applicative	<code>curl</code> porteur de <code>union select</code>	Alerte SID 1000003 dans <code>fast.log</code>
Détection réseau	<code>nmap -p 1194 -sU 192.168.1.17</code> , répété	Alerte SID 1000002
Journalisation exploitable	Requête <code>jq</code> sur <code>eve.json</code>	L'alerte au format structuré, URL comprise
Blocage effectif	<code>curl</code> sur <code>/install/</code> après bascule en IPS	Pas de réponse, et <code>[Drop]</code> dans <code>fast.log</code>

Captures à produire lors du déploiement, à numéroter à la suite des captures existantes — soit à partir de 25- — et à reporter dans la table des figures de l'annexe F :

- **InstallationSuricata** — la sortie de `suricata --build-info`, version visible
- **ConfigurationSuricataInterfaces** — le bloc `af-packet` avec les trois interfaces, et le bloc `HOME_NET`
- **TestConfigurationSuricata** — la ligne « Configuration provided was successfully loaded »
- **ReglesLocales** — le contenu de `local.rules`
- **DeclenchementRegle** — le `curl` lancé depuis le poste client, qui sert de repère horaire
- **AlerteFastLog** — la ligne d'alerte, avec son SID, l'adresse source et l'horodatage
- **AlerteEveJson** — la même alerte mise en forme par `jq`
- **ModeBlocage** — la sortie de `iptables -L DOCKER-USER` et une ligne `[Drop]`

Placer **DeclenchementRegle** et **AlerteFastLog** côte à côte : la correspondance des horodatages fait la démonstration en une image, ce qu'aucun commentaire ne remplace.

3.8.7 Passage en mode prévention

Une fois les alertes constatées, et pas avant, la bascule en IPS insère Suricata dans la chaîne de traitement des paquets par **NFQUEUE** : chaque paquet lui est soumis, et il décide de son sort.

Dans `/etc/default/suricata` :

```
LISTENMODE=nfqueue
NFQUEUE=0
```

La dérivation vers la file s'ajoute **après** la règle d'isolation du paragraphe 3.6, dont l'ordre ne doit pas être modifié :

```
sudo iptables -A DOCKER-USER -j NFQUEUE --queue-num 0 --queue-bypass
sudo iptables -I INPUT 1 -j NFQUEUE --queue-num 0 --queue-bypass

sudo iptables -L DOCKER-USER -n --line-numbers
sudo systemctl restart suricata
```

--queue-bypass n'est pas une option de confort

Sans ce mot-clé, l'arrêt du processus Suricata fait tomber **la totalité du trafic de la machine** : plus de tunnel, plus de SSH, plus aucun moyen de se rattraper autrement qu'à la console de l'hyperviseur. Avec lui, le trafic passe normalement quand l'IPS est absent.

C'est un arbitrage assumé entre disponibilité et sécurité : sur cette maquette, la disponibilité l'emporte, un IPS en coupure étant un point de défaillance unique. En exploitation on lui préfère plusieurs files et une supervision du service — mais jamais l'absence de `--queue-bypass` sans accès console garanti.

Ordre des règles dans DOCKER-USER : à reconstruire après redémarrage

La règle d'isolation du paragraphe 3.6.1 est posée par une unité **systemd** au démarrage, après Docker ; la dérivation **NFQUEUE** ci-dessus est, elle, rendue persistante par **netfilter-persistent**. **Deux mécanismes de restauration agissent donc sur la même chaîne**, avec un risque de règle en double ou d'ordre inversé.

L'ordre correct est : **DROP** sur **enp0s8** en position 1, dérivation **NFQUEUE** ensuite. Inversé, les paquets venus du réseau local sont soumis à l'IPS au lieu d'être rejetés d'emblée — le service reste protégé, mais la sonde traite un trafic qu'elle n'a pas à voir, et la lecture du jeu de règles devient trompeuse. Contrôler après chaque redémarrage avec **iptables -L DOCKER-USER -n --line-numbers**.

Transformer ensuite les règles à bloquer **une à la fois**, en revalidant après chacune. Passer l'ensemble du jeu en **drop** d'un seul coup rend tout diagnostic impossible : une coupure de service ne se rattache plus à une règle identifiable.

```
sudo sed -i 's/^alert http \(.*LAB acces \/install\)\/drop http \1/' \
/var/lib/suricata/rules/local.rules

sudo suricata -T -c /etc/suricata/suricata.yaml -v
sudo systemctl restart suricata

# rendre le jeu de regles reseau persistant
sudo netfilter-persistent save
```

Vérification du blocage. Depuis le poste client, le **curl** sur **/install/** ne reçoit plus de réponse, et **fast.log** affiche **[Drop]** en place de **[**]**. C'est le seul contrôle qui atteste du passage en prévention : la présence de la règle **NFQUEUE** dans **iptables** montre que le trafic est dérivé, pas qu'il est effectivement bloqué.

3.8.8 Dépannage

Symptôme	Cause probable	Correction
Suricata tourne mais aucune alerte n'apparaît	Interface inexistante dans af-packet — le fameux eth0	Corriger avec les noms relevés au 3.8.2, revalider avec -T
Les règles applicatives ne se déclenchent jamais	Sonde posée seulement sur enp0s8 ou tun0 , où le trafic est chiffré	Ajouter l'interface br-... du réseau Docker
Le trafic entre NGINX et GLPI est vu comme externe	Sous-réseau Docker absent de HOME_NET	L'ajouter, puis redémarrer le service
Le service refuse de démarrer	Erreur de syntaxe YAML, ou règle invalide	suricata -T -c ... -v désigne la ligne fautive
La machine est injoignable après bascule en IPS	--queue-bypass oublié	Console de l'hyperviseur, puis iptables -F INPUT et refaire la règle
Le serveur rame, la VM sature	Jeu de règles complet trop lourd pour 4 Go de RAM	Désactiver des catégories via suricata-update , ou passer detect: profile: low dans suricata.yaml
Les règles réseau disparaissent au redémarrage	netfilter-persistent save non exécuté après modification	Le relancer, puis contrôler l'ordre des règles (encadré ci-dessus)

4. Exploitation courante

4.1 Commandes de base

Toutes les commandes **docker compose** se lancent depuis **~/ticketing-projet/glpi**, où se trouve le fichier de composition.

```
cd ~/ticketing-projet/glpi

sudo docker compose ps          # etat des trois conteneurs
sudo docker compose up -d       # demarrer, ou appliquer une modification
sudo docker compose restart nginx # redemarrer un seul service
sudo docker compose stop        # arreter sans supprimer
sudo docker compose down        # arreter et supprimer conteneurs et reseau
                                # (les volumes nommes sont conserves)
docker stats --no-stream        # consommation memoire et processeur
```

4.2 Journalisation et diagnostic

```
sudo docker compose logs -f --tail=100    # les trois services
sudo docker compose logs -f nginx         # un seul service
sudo docker exec glpi_proxy nginx -t      # valide de la configuration NGINX
sudo journalctl -u openvpn-server@server -f # tunnel VPN
sudo ufw status verbose                   # regles du pare-feu
sudo iptables -L DOCKER-USER -n --line-numbers # isolation des conteneurs
sudo docker exec glpi_web php /var/www/html/glpi/front/cron.php # actions automatiques (2.10)
sudo tail -n 20 /var/log/suricata/fast.log # alertes de la sonde (3.8)
```

Arbre de décision pour un client qui ne joint pas GLPI :

Symptôme	Vérification	Cause probable
Aucune réponse, délai d'attente dépassé	<code>ip addr show tun0</code> sur le poste client	Le tunnel n'est pas monté – c'est le cas le plus fréquent
Le tunnel est monté mais aucune réponse	<code>ping 10.8.0.1</code> depuis le client	Règle <code>allow in on tun0</code> absente, ou service OpenVPN arrêté
<code>ping</code> passe mais HTTPS échoue	<code>sudo docker compose ps</code> sur le serveur	Conteneur <code>glpi_proxy</code> arrêté
Erreur 502 renvoyée par NGINX	<code>sudo docker compose logs glpi</code>	<code>glpi_web</code> est arrêté ou en cours de démarrage
GLPI signale une erreur de base de données	<code>sudo docker compose logs db</code>	<code>glpi_db</code> est arrêté, ou volume <code>glpi_db_data</code> corrompu
Le service répond depuis le réseau local	<code>sudo iptables -L DOCKER-USER -n</code>	Anomalie de sécurité : la règle d'isolation a disparu, typiquement après un redémarrage. Appliquer le paragraphe 3.6.1

4.3 Sauvegarde

Aucune sauvegarde n'est en place à ce jour. Le script ci-dessous couvre les trois éléments dont la perte serait irréversible : la base, les fichiers de l'application, la configuration et les secrets.

```

sudo tee /usr/local/sbin/sauvegarde-glpi.sh > /dev/null <<'EOF'
#!/bin/bash
set -euo pipefail

DEST=/var/backups/glpi
STAMP=$(date +%F)
PROJET=/home/ubuntu/ticketing-projet
mkdir -p "$DEST"

# 1. base de donnees : --single-transaction assure un dump coherent
#   sans verrouiller les tables. Le mot de passe est lu dans
#   l'environnement du conteneur, il n'apparaît ni dans la ligne
#   de commande ni dans l'historique du shell.
docker exec glpi_db sh -c \
'exec mariadb-dump --single-transaction -u root -p"$MYSQL_ROOT_PASSWORD" glpidb' \
| gzip > "$DEST/glpidb-$STAMP.sql.gz"

# 2. fichiers de l'application : documents joints, greffons, configuration
docker run --rm -v glpi_glpi_data:/data -v "$DEST":/backup alpine \
tar czf "/backup/glpi_data-$STAMP.tar.gz" -C /data .

# 3. configuration, secrets et certificats
tar czf "$DEST/config-$STAMP.tar.gz" -C "$PROJET" \
glpi/.env glpi/docker-compose.yml nginx/

# 4. rotation : conserver quatorze jours
find "$DEST" -name '*.gz' -mtime +14 -delete
EOF

sudo chmod 700 /usr/local/sbin/sauvegarde-glpi.sh

```

Planification quotidienne à 2 h 00 :

```

echo '0 2 * * * /usr/local/sbin/sauvegarde-glpi.sh >> /var/log/sauvegarde-glpi.log 2>&1' \
| sudo tee /etc/cron.d/sauvegarde-glpi > /dev/null
sudo chmod 644 /etc/cron.d/sauvegarde-glpi

```

Une sauvegarde non testée n'est pas une sauvegarde

Deux exigences complètent le script. **Copier les archives hors de la machine virtuelle** : conservées sur le serveur, elles disparaissent avec lui, ce qui est précisément le scénario contre lequel on se protège. Et **rejouer une restauration complète au moins une fois**, en suivant le paragraphe 4.4 : c'est le seul moyen de savoir que les archives sont exploitables. Noter enfin que **config-*.tar.gz** contient le fichier **.env** en clair et doit être protégé en conséquence.

4.4 Restauration

Procédure destructive

Les commandes ci-dessous **suppriment les volumes existants** avant de réinjecter les données. À n'exécuter que sur un serveur dont l'état actuel est perdu ou volontairement abandonné.

```

cd ~/ticketing-projet/glpi
STAMP=AAAA-MM-JJ          # date de l'archive a restaurer

# 1. arreter la pile et repartir de volumes vierges
sudo docker compose down
docker volume rm glpi_glpi_data glpi_db_data

# 2. demarrer la seule base, et attendre qu'elle accepte les connexions
sudo docker compose up -d db
until docker exec glpi_db mariadb-admin ping --silent 2>/dev/null; do sleep 2; done

# 3. reinjecter le dump
zcat "/var/backups/glpi/glpidb-$STAMP.sql.gz" \
| docker exec -i glpi_db sh -c 'exec mariadb -u root -p"$MYSQL_ROOT_PASSWORD" glpidb'

# 4. restaurer les fichiers de l'application
docker run --rm -v glpi_glpi_data:/data -v /var/backups/glpi:/backup alpine \
tar xzf "/backup/glpi_data-$STAMP.tar.gz" -C /data

# 5. remonter l'ensemble et verifier
sudo docker compose up -d
sudo docker compose ps
curl -Ik https://localhost/

```

Après restauration, contrôler dans l'interface que le nombre de tickets et d'éléments de parc correspond à l'attendu, et que les pièces jointes d'un ticket connu s'ouvrent correctement – ce second point valide la restauration du volume `glpi_glpi_data`, qu'un simple décompte ne couvre pas.

4.5 Renouvellement du certificat TLS

Le certificat est valable **365 jours** à compter de sa génération, et rien n'alerte de son expiration : passé l'échéance, les navigateurs refuseront la connexion. Relever l'échéance et la porter à l'agenda d'exploitation.

```

# relever la date d'expiration
openssl x509 -in ~/ticketing-projet/nginx/certs/glpi.crt -noout -enddate

# renouveler (memes parametres, subjectAltName inclus)
cd ~/ticketing-projet/nginx
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout certs/glpi.key -out certs/glpi.crt \
-subj "/C=FR/ST=Loire/L=Nantes/O=MonEntreprise/OU=IT/CN=glpi.local" \
-addext "subjectAltName=DNS:glpi.local,IP:192.168.1.17,IP:10.8.0.1"
sudo chmod 600 certs/glpi.key

# recharger NGINX : un restart suffit, les fichiers sont montes depuis l'hote
cd ~/ticketing-projet/glpi
sudo docker compose restart nginx

```

Surveillance automatique de l'échéance, avec alerte à trente jours :

```

echo '0 8 * * 1 openssl x509 -in /home/ubuntu/ticketing-projet/nginx/certs/glpi.crt \
-noout -checkend 2592000 || echo "Certificat GLPI : expiration dans moins de 30 jours" ' \
| sudo tee /etc/cron.d/verif-cert-glpi > /dev/null

```


4.6 Mises à jour

```
# systeme d'exploitation
sudo apt update && sudo apt upgrade -y

# images des conteneurs
cd ~/ticketing-projet/glpi
sudo /usr/local/sbin/sauvegarde-glpi.sh      # IMPERATIF avant toute mise a jour
sudo docker compose pull
sudo docker compose up -d
sudo docker image prune -f

# apres toute mise a jour de l'image GLPI, refaire :
sudo docker exec glpi_web rm -f /var/www/html/glpi/install/install.php
sudo iptables -L DOCKER-USER -n --line-numbers  # controler l'isolation
```

Les étiquettes `latest` sont un risque de mise à jour non maîtrisée

`diouxx/glpi:latest` et `nginx:latest` ne désignent pas une version : un `docker compose pull` peut donc livrer une **version majeure** de GLPI, avec migration de schéma de base irréversible et greffons rendus incompatibles. Deux conséquences pratiques : ne jamais lancer `pull` sans sauvegarde vérifiée, et remplacer ces étiquettes par des versions explicites – à l'image de `mariadb:10.11`, qui est correctement épinglé.

5. État de conformité au cahier de recettes

Ce chapitre situe l'état du service par rapport aux exigences du cahier de recettes, et décrit la voie de mise en œuvre de ce qui reste à faire. Il complète le registre d'écarts du chapitre 6, qui porte sur les défauts de l'existant.

5.1 Tableau de conformité

Exigence du cahier de recettes	Priorité	État	Preuve	Renvoi
Environnement réseau : 2 machines, serveur et client	Très haute	Partiel	Fig. 1, 16	Serveur opérationnel ; le poste client existe comme client du tunnel, son paramétrage n'est pas attesté – voir 5.2
Serveur GLPI sur Ubuntu Server, 4 Go de RAM	Haute	Conforme	Fig. 5 à 9	Chapitre 2
Ajout d'un VPN pour se connecter à GLPI	Haute	Conforme	Fig. 16	3.4 ; mise en place manuelle de la PKI en 3.4.4
Sécurisation du tunnel : n'autoriser que le VPN en entrée	Haute	Conforme	Fig. 17, 18	3.5 et 3.6 – non persistant , voir 3.6.1
Ajout d'un pare-feu pour protéger le réseau	Haute	Conforme	Fig. 17	3.5
Chiffrement des flux (<i>ajout hors cahier</i>)	–	Conforme	Fig. 11 à 15	3.1 à 3.3 – certificat auto-signé
Client et agent GLPI pour l'inventaire	Haute	À faire	–	5.2
Script de collecte des mails vers GLPI	Haute	Conforme	Fig. 19 à 21	2.10 – collecteur natif de GLPI, aucun script spécifique écrit
Outil externe de contrôle à distance	Moyenne	À faire	–	5.4
Double authentification (MFA)	Haute	À faire	–	5.5
IPS Suricata	Haute	À faire	–	3.8 – procédure établie et adaptée à l'architecture, déploiement à réaliser

L'ensemble de la chaîne de sécurisation réseau demandée est en place et vérifiée : tunnel VPN, pare-feu, isolation des conteneurs, chiffrement du transport. Ce qui reste relève pour l'essentiel de paramétrage applicatif dans GLPI – trois des cinq points ci-dessous sont des fonctions natives de la version 10, et non des développements.

5.2 Poste client et agent d'inventaire

GLPI 10 intègre l'inventaire : aucune extension de type FusionInventory n'est requise, comme le rappelle le cahier de recettes. La mise en œuvre tient en trois points : activer l'inventaire côté serveur dans

Administration > Inventaire, installer le GLPI Agent sur le poste Ubuntu client, puis le faire pointer sur l'URL du serveur.

Deux points d'attention propres à cette architecture. L'agent doit joindre le serveur à travers le tunnel, donc via son adresse sur **tun0** et non son adresse locale, sans quoi la règle **DOCKER-USER** bloquera ses remontées. Et l'agent vérifie le certificat TLS : face à un certificat auto-signé, lui distribuer l'autorité de certification plutôt que désactiver la vérification, option toujours tentante et toujours mauvaise.

5.3 Collecte des courriels et création automatisée de tickets

Le cahier de recettes évoque un « *script* » et invite à regarder ceux qui existent déjà. La bonne réponse est qu'aucun script n'est nécessaire : GLPI dispose d'un **récepteur de courriels** natif, à configurer dans *Configuration > Récepteurs de courriels*. Il faut une boîte dédiée, un accès IMAPS en 993/tcp, et l'activation de l'action automatique **mailgate** – typiquement toutes les cinq minutes.

Côté réseau, la politique **default allow outgoing** déjà en place autorise cette sortie : aucune règle supplémentaire à prévoir. Côté exploitation, deux garde-fous méritent d'être posés d'entrée : un filtrage anti-spam en amont, faute de quoi chaque message indésirable crée un ticket, et une règle de rejet des messages automatiques pour éviter les boucles de notification.

5.4 Outil de contrôle à distance

Le cahier de recettes propose VNC ou TeamViewer. **Ces deux options ne sont pas équivalentes au regard de l'architecture retenue.** TeamViewer fonctionne en établissant une connexion sortante vers un relais hébergé chez l'éditeur : le poste devient joignable de l'extérieur par un canal qui échappe intégralement au pare-feu et au VPN, ce qui contredit directement la consigne « n'autoriser que le tunnel du VPN en entrée ».

La solution cohérente est un serveur VNC (**x11vnc** ou TightVNC) installé sur le poste client, écoutant **uniquement sur l'interface du tunnel**, le technicien s'y connectant après avoir monté le VPN. VNC ne chiffrant pas nativement, le tunnel fournit ici le chiffrement qui lui manque : l'architecture rend donc l'option la plus simple aussi la plus sûre. Un lien vers la session peut ensuite être ajouté dans la fiche de l'ordinateur côté GLPI.

5.5 Double authentification

Niveau	Mise en œuvre	Priorité
Application GLPI	TOTP native dans GLPI 10 (<i>Configuration > Authentification</i>), à imposer au minimum aux profils Super-Admin et Technicien	Haute
Accès SSH	libpam-google-authenticator , en conservant impérativement une session ouverte pendant la mise en place pour ne pas se verrouiller dehors	Moyenne
Tunnel VPN	Ajout d'un second facteur à l'authentification par certificat via auth-user-pass couplé à un module PAM TOTP	Moyenne

La priorité va sans hésitation à GLPI : c'est là que se trouvent les données, et c'est la seule des trois couches dont l'accès repose aujourd'hui sur un simple mot de passe.

5.6 Système de détection et de prévention d'intrusion Suricata

Le cahier de recettes signale d'emblée le point délicat : « *Attention aux fichiers de configuration* ». Deux décisions structurent le déploiement.

Mode de fonctionnement. En mode IDS, Suricata observe et alerte ; en mode IPS, il bloque, en s'insérant dans la chaîne de traitement des paquets via `NFQUEUE`. Le cahier de recettes demande un IPS, ce qui suppose de coordonner la règle `NFQUEUE` avec les règles `iptables` déjà posées – dont celle de la chaîne `DOCKER-USER` – sous peine de conflits d'ordre d'évaluation ou de coupure du service.

Points de configuration. Dans `suricata.yaml`, `HOME_NET` doit déclarer les deux réseaux réellement en jeu, `10.8.0.0/24` pour le tunnel et `192.168.1.0/24` pour le réseau local ; les interfaces à surveiller sont `tun0` et `enp0s8` ; le jeu de règles s'installe et se met à jour avec `suricata-update` (règles ET Open).

La limite à anticiper : le chiffrement aveugle la sonde

Le trafic GLPI est chiffré de bout en bout par NGINX, et le trafic VPN l'est aussi. Suricata ne verra donc **ni les requêtes HTTP, ni le contenu du tunnel** : les règles de détection applicative web seront inopérantes. Trois orientations restent pertinentes : surveiller ce qui demeure visible (balayages de ports, tentatives de connexion répétées sur 1194/udp et 22/tcp, anomalies TLS), placer la sonde en aval du proxy sur le réseau des conteneurs où le trafic redevient clair, ou activer la journalisation TLS pour détecter certificats et destinations suspects. Un IPS posé sans cette analyse préalable donnerait un sentiment de couverture largement illusoire.

6. Registre des points de vigilance

État des écarts connus sur l'existant à la date de rédaction, classés par priorité décroissante. Ce registre est destiné à être tenu à jour au fil des interventions.

#	Constat	Conséquence	Action	Réf.
1	La règle <code>iptables</code> d'isolation des conteneurs n'est pas persistante	L'isolation disparaît au redémarrage, sans signal visible : GLPI redevient joignable hors VPN	Unité <code>systemd docker-isolation</code>	3.6.1
2	Aucune sauvegarde n'est planifiée	La perte de la machine virtuelle entraîne celle du parc et de l'historique des tickets	Script et tâche planifiée	4.3
3	Les comptes <code>tech</code> , <code>normal</code> et <code>post-only</code> conservent leur mot de passe par défaut	Identifiants publiquement documentés, testés par les outils de balayage	Changer ou supprimer ces comptes	2.8, 3.7
4	Les données de démonstration sont toujours actives	5 400 ordinateurs et 1 500 tickets fictifs faussent tout indicateur	Désactiver depuis le tableau de bord	2.8
5	Le certificat TLS est auto-signé et sans <code>subjectAltName</code>	Avertissement permanent du navigateur, qui habitue les utilisateurs à passer outre les alertes	Certificat d'autorité interne, ou a minima régénérer avec <code>-addext</code>	3.1
6	Les certificats VPN sont valables 3 650 jours	Fenêtre de compromission de dix ans	Réémettre sur une validité d'un an	3.4.1
7	Les images utilisent l'étiquette <code>latest</code>	Un <code>pull</code> peut livrer une version majeure et migrer la base de manière irréversible	Épingler des versions explicites	4.6
8	Aucune double authentification n'est active	L'accès à l'ensemble des données du parc ne dépend que d'un mot de passe	Activer le TOTP natif de GLPI	5.5
9	L'échéance du certificat n'est pas surveillée	Interruption de service brutale à l'expiration	Tâche planifiée <code>checkend</code>	4.5
10	Le fichier <code>.env</code> a été affiché en clair lors du déploiement	Mots de passe de la base à considérer comme divulgués	Régénérer les mots de passe et exclure <code>.env</code> du versionnement	2.4
11	Le collecteur de courriels se connecte avec l'option <code>novalidate-cert</code>	La session IMAP est chiffrée mais le certificat du serveur n'est pas vérifié : l'authentification du serveur de messagerie est perdue	Décocher <code>NO-VALIDATE-CERT</code> et contrôler que la relève fonctionne toujours	2.10.2
12	La tâche planifiée du collecteur emploie un chemin non confirmé et une redirection non	Une tâche planifiée échoue en silence : sans ticket créé, rien ne signale la panne	Confirmer le chemin de <code>cron.php</code> dans le conteneur, écrire <code>>/dev/null 2>&1</code>	2.10.4, annexe D

	portable			
13	Trois contrôles du collecteur ne sont pas passés, et le filtrage des indésirables n'est pas en place	Un doublon ou une réponse non rattachée resterait invisible ; la boîte relevée sert aussi à émettre les notifications, ce qui expose à une boucle	Passer les tests restants du 2.10.5, filtrer en amont et rejeter les auto-réponses	2.10.5, 2.10.6
14	Aucun système de détection d'intrusion n'est déployé	Une tentative d'intrusion ne laisse aucune trace exploitable : ni alerte, ni journal d'attaque	Appliquer la procédure du paragraphe 3.8, sonde sur les trois interfaces	3.8, annexe E

Annexe A. docker-compose.yml

Fichier de référence, tel que versionné dans le dépôt. Emplacement sur le serveur : `~/ticketing-projet/glpi/docker-compose.yml`.

```
version: '3.8'

services:
  db:
    image: mariadb:10.11
    container_name: glpi_db
    restart: always
    environment:
      - MYSQL_ROOT_PASSWORD=${MYSQL_ROOT_PASSWORD}
      - MYSQL_DATABASE=${MYSQL_DATABASE}
      - MYSQL_USER=${MYSQL_USER}
      - MYSQL_PASSWORD=${MYSQL_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  glpi:
    image: diouxx/glpi:latest
    container_name: glpi_web
    restart: always
    environment:
      - TIMEZONE=Europe/Paris
    depends_on:
      - db
    volumes:
      - glpi_data:/var/www/html/glpi

  nginx:
    image: nginx:latest
    container_name: glpi_proxy
    restart: always
    ports:
      - '80:80'
      - '443:443'
    volumes:
      - ../nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
      - ../nginx/certs:/etc/nginx/certs:ro
    depends_on:
      - glpi

volumes:
  db_data:
  glpi_data:
```

L'attribut `version: '3.8'` de la première ligne est ignoré par Docker Compose v2 et provoque un avertissement à chaque commande. Il peut être supprimé sans conséquence.

Annexe B. nginx.conf

Emplacement sur le serveur : `~/ticketing-projet/nginx/nginx.conf`, monté dans le conteneur sur `/etc/nginx/conf.d/default.conf` en lecture seule.

```

server {
    listen 80;
    server_name _;
    # On force la redirection de HTTP vers HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name _;

    # Emplacement des certificats qu'on vient de créer
    ssl_certificate /etc/nginx/certs/glpi.crt;
    ssl_certificate_key /etc/nginx/certs/glpi.key;

    # Paramètres de sécurité de base
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        # On renvoie le trafic vers le conteneur GLPI sur son port 80 interne
        proxy_pass http://glpi:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Annexe C. Jeu de règles réseau de référence

État attendu du filtrage après application de l'ensemble des procédures, correctif de persistance inclus. À comparer à la sortie réelle lors d'un contrôle.

```

# ---- UFW -----
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp          # administration SSH
sudo ufw allow 1194/udp        # entree du tunnel OpenVPN
sudo ufw allow in on tun0      # tout ce qui vient du tunnel
sudo ufw enable
# NB : 443/tcp n'est volontairement PAS ouvert.

# ---- iptables : isolation des conteneurs -----
sudo iptables -I DOCKER-USER -i enp0s8 -j DROP
# rendue persistante par l'unité systemd docker-isolation.service

# ---- controles -----
sudo ufw status verbose
sudo iptables -L DOCKER-USER -n -v --line-numbers
curl -Ik --max-time 5 https://192.168.1.17/ # hors tunnel : doit ECHOUER
curl -Ik --max-time 5 https://10.8.0.1/     # dans le tunnel : doit REPONDRE

```

Annexe D. Collecteur de courriels : planification et contrôles

Tâche planifiée de l'hôte qui sert les actions automatiques de GLPI, dont **mailgate**. Versionnée dans le dépôt sous **serveur/scrcriptCron.txt**. La forme corrigée des deux réserves du paragraphe 2.10.4 est donnée en second.

```

# --- en place sur le serveur -----
* * * * * docker exec glpi_web php /var/www/html/front/cron.php &> /dev/null

# --- forme corrigee : chemin a confirmer, redirection portable ---
* * * * * docker exec glpi_web php /var/www/html/glpi/front/cron.php >/dev/null 2>&1

```

Contrôles du collecteur, à passer dans cet ordre :


```
# 1. l'extension PHP imap est presente dans le conteneur
sudo docker exec glpi_web php -m | grep -i imap

# 2. le chemin de cron.php dans l'image
sudo docker exec glpi_web ls /var/www/html/glpi/front/cron.php

# 3. execution manuelle, sans redirection : revele toute erreur
sudo docker exec glpi_web php /var/www/html/glpi/front/cron.php

# 4. la tache planifiee est bien en place
sudo crontab -l ; ls -l /etc/cron.d/
```

Côté interface : **Configuration > Collecteurs** pour la fiche du collecteur, **Configuration > Actions automatiques > mailgate** onglets *Journaux* et *Statistiques* pour le nombre de messages traités à chaque passage, et **Assistance > Tickets** filtré sur la source *E-Mail* pour la liste des tickets réellement issus de la relève.

Annexe E. Suricata : jeu de règles local

Une règle par service exposé, à déposer dans `/var/lib/suricata/rules/local.rules` et à déclarer dans `rule-files` — voir le paragraphe 3.8.4. Les règles 1000003 à 1000005 ne produisent d'alerte que si la sonde écoute l'interface du réseau Docker, seul endroit où l'URL circule en clair.

```
# /var/lib/suricata/rules/local.rules
# Numeros pris a partir de 1000000 : plage reservee aux regles locales.

# SSH - tentative de force brute
alert tcp $EXTERNAL_NET any -> $HOME_NET 22 (msg:"LAB SSH force brute"; \
  flow:to_server; flags:S; threshold:type both, track by_src, count 5, seconds 60; \
  classtype:attempted-recon; sid:1000001; rev:1;)

# VPN - balayage du port OpenVPN
alert udp $EXTERNAL_NET any -> $HOME_NET 1194 (msg:"LAB scan port OpenVPN"; \
  threshold:type both, track by_src, count 10, seconds 60; \
  classtype:attempted-recon; sid:1000002; rev:1;)

# GLPI - tentative d'injection SQL (visible sur le reseau Docker, en clair)
alert http any any -> $HOME_NET any (msg:"LAB GLPI injection SQL suspectee"; \
  flow:to_server,established; http.uri; content:"union"; nocase; \
  content:"select"; nocase; distance:0; \
  classtype:web-application-attack; sid:1000003; rev:1;)

# GLPI - acces au repertoire d'installation
alert http any any -> $HOME_NET any (msg:"LAB acces /install detecte"; \
  flow:to_server,established; http.uri; content:"/install/"; nocase; \
  classtype:web-application-attack; sid:1000004; rev:1;)

# GLPI - force brute sur le formulaire de connexion
alert http any any -> $HOME_NET any (msg:"LAB GLPI force brute connexion"; \
  flow:to_server,established; http.method; content:"POST"; \
  http.uri; content:"/front/login.php"; nocase; \
  threshold:type both, track by_src, count 10, seconds 60; \
  classtype:attempted-user; sid:1000005; rev:1;)
```

Après toute modification du fichier, revalider puis redémarrer — dans cet ordre, jamais l'inverse :

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
sudo systemctl restart suricata
```

Passage d'une règle en blocage

Remplacer `alert` par `drop` en tête de la règle concernée, **une règle à la fois**, en revalidant après chacune. Le mode `nfqueue` doit être actif (§3.8.7), faute de quoi `drop` reste sans effet : Suricata signale le paquet mais n'a pas la main pour le supprimer. Une règle en `drop` sur une sonde en mode IDS est une protection imaginaire.

Annexe F. Table des figures

Correspondance entre les figures de cette documentation et les fichiers de capture d'origine, disponibles dans le répertoire [capture/](#) du dépôt.

Figure	Fichier source	Objet
1	1-MiseAJourVM.PNG	Résultat attendu : la liste des paquets est actualisée et la mise à niveau démarre
2	2-InstallationDocker.PNG	Installation du moteur Docker et de Docker Compose
3	3-EtatDocker.PNG	Contrôle de la version
4	4-FichierEnvAvecMdpMYSQL.PNG	Structure du fichier .env telle que créée lors du déploiement initial
5	5-EcritureDockerCompsoe.PNG	Fichier de composition initial : MariaDB et GLPI
6	6-LancementDockerCompose.PNG	Résultat attendu : création du réseau et des volumes, téléchargement des images, puis démarrage des...
7	7-EtatImagesGLPI.PNG	Les deux conteneurs sont à l'état Up. glpi_db n'expose 3306 qu'en interne, tandis que glpi_web publie...
8	8-ConfigurationGLPI.PNG	Étape 6 : l'installation est terminée
9	9-PremiereConnexionGLPI.PNG	Première connexion
10	10-ModificaionMdpParDefaut.PNG	Changement du mot de passe du compte Super-Admin glpi
11	9-GenerationDuCertificatSSL.PNG (recadrée)	Génération du certificat auto-signé, RSA 2048 bits, valable 365 jours
12	11-ConfigurationNGINX.PNG	Configuration du reverse proxy
13	12-AjoutNGINXDansDockerCompose.PNG	Fichier de composition final
14	13-LancementAvecNGINX.PNG	Résultat attendu : quatre éléments démarrés – le réseau et les trois conteneurs
15	14-AccessGLPIEnHTTPS.PNG	GLPI répond en HTTPS
16	15-InstallationOpenVPN.PNG	Installation en mode non interactif
17	20-ConfigurationPareFeuServeur.PNG	Mise en place du jeu de règles
18	21-BlocgaeTraficVersDocker.PNG	Blocage de tout trafic à destination des conteneurs arrivant par l'interface physique
19	22-CollecteurMail.PNG	Fiche du collecteur « Collecteur gmail » et sa chaîne de connexion
20	23-AutomatisationCollecteur.PNG	L'action automatique mailgate en mode CLI, fréquence de cinq minutes
21	24-RecuperationDeTickets.PNG	Ticket créé depuis un courriel : source E-Mail, demandeur reconnu